SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

ZAVRŠNI RAD

Personalizirana aplikacija za planiranje putovanja

Roko Šabić

Zagreb, lipanj 2025.

Sadržaj

Uvod

1. Analiza reprezentativnih aplikacija
    1.1. TripIt
    1.2. Google Travel
    1.3. Wanderlog
2. Plan projekta
    2.1. Funkcionalni zahtjevi
    2.2. Nefunkcionalni zahtjevi
    2.3. Ciljana skupina korisnika
    2.4. Glavni problemi korisnika
    2.5. Tehnološka ograničenja
    2.6. Rizici
3. Arhitektura sustava
    3.1. Osnovna arhitektura sustava
    3.2. Baza podataka
    3.3. Arhitektura poslužitelja
        3.3.1. Next.js API rute
        3.3.2. Interakcija s bazom podataka
        3.3.3. Autentifikacija
        3.3.4. Vanjske API integracije
        3.3.5. Okruženje i postavljanje
    3.4. Arhitektura korisničkog sučelja
        3.4.1. Next.js App Router i Navigacija
        3.4.2. React komponente
        3.4.3. Upravljanje stanjem
        3.4.4. Stiliziranje
        3.4.5. Dohvaćanje podataka
        3.4.6. Tipovi i uslužne funkcije
        3.4.7. Korisničko iskustvo (UX) i UI obrasci
4. Funkcionalnosti aplikacije

4.1. Autentifikacija
4.2. Putovanja
4.2.1. Stvaranje putovanja
4.2.2. Pregled detalja putovanja
4.3. Tečajevi valuta
5. Korištene tehnologije
5.1. Next.js
5.2. Vercel
5.3. TypeScript
5.4. Tailwind CSS
5.5. Supabase
5.6. Visual Studio Code
5.7. GitHub
5.8. Kaggle
5.9. Pandas (Python)
5.10. Google Places API
5.11. ExchangeRate API
Zaključak
Literatura
Sažetak
Summary

UVOD

Planiranje putovanja, osobito onih koja uključuju više međusobno povezanih aktivnosti poput rezervacije smještaja i letova, upravljanja financijama te pripreme itinerara, često predstavlja složen i zahtjevan zadatak za korisnike. U suvremenom digitalnom okruženju, sve je izraženija potreba za integriranim rješenjima koja omogućuju objedinjavanje svih relevantnih informacija i funkcionalnosti u okviru jedinstvene aplikacije. U tom kontekstu razvijen je sustav za digitalno upravljanje putovanjima koji korisnicima omogućuje sveobuhvatan nadzor nad različitim aspektima putovanja – od logistike i budžetiranja do informiranja i personalizacije sadržaja.

Svrha ovog sustava je olakšati krajnjim korisnicima planiranje i realizaciju putovanja kroz centraliziranu i pristupačnu digitalnu platformu. Sustav nudi niz funkcionalnosti koje uključuju registraciju i autentifikaciju korisnika, rezervaciju letova i smještaja, upravljanje osobnim budžetom, praćenje i evidentiranje troškova, pregled vremenske prognoze, konverziju valuta, organizaciju popisa za pakiranje, te pregled i označavanje znamenitosti kao favorita. Svaka od navedenih komponenti doprinosi smanjenju kognitivnog opterećenja korisnika te omogućuje bolje planiranje i efikasnije provođenje putovanja.

Poseban naglasak stavljen je na funkcionalnosti koje se odnose na financijsko upravljanje. Korisnik ima mogućnost definiranja početnog budžeta za putovanje, nakon čega može unositi pojedinačne troškove te na taj način pratiti ukupan iznos potrošnje i usporediti ga s početnim planom. Kroz integraciju s vanjskim servisima za dohvat aktualnih deviznih tečajeva, sustav omogućuje i jednostavnu konverziju valuta, što je osobito korisno pri međunarodnim putovanjima. Također, implementiran je modul za praćenje vremenskih uvjeta koji korisniku omogućuje uvid u meteorološku prognozu za određenu destinaciju tijekom odabranog vremenskog razdoblja.

U okviru aplikacije razvijen je i sustav za izradu popisa potrebnih stvari za pakiranje, čime se korisniku pruža organizacijska pomoć u pripremi za putovanje. Uz to, sustav omogućuje pregled popularnih turističkih atrakcija i znamenitosti unutar određene destinacije, kao i njihovo filtriranje po tematskim kategorijama. Korisnik može spremiti atrakcije u vlastiti popis favorita te ih kasnije jednostavno pregledavati putem interaktivne karte.

Uloga administratora u sustavu je višestruka. Administratori imaju uvid u aktivnosti korisnika, pristup svim kreiranim putovanjima, kao i mogućnost uređivanja i brisanja podataka. Time je osigurana nadzorna funkcija unutar sustava te se omogućuje održavanje podatkovne konzistentnosti i sprječavanje zlouporabe.

Tehnička implementacija sustava temelji se na modernim web tehnologijama, pri čemu se koriste Next.js s podrškom za TypeScript za razvoj korisničkog sučelja, dok se za pohranu podataka koristi PostgreSQL. Supabase platforma iskorištena je za autentifikaciju,

administraciju baze podataka te razvoj sigurnog API-ja. Sustav je razvijen u skladu s načelima modularnosti, skalabilnosti i sigurnosti, čime se osigurava dugoročna održivost i mogućnost daljnjeg proširenja funkcionalnosti.

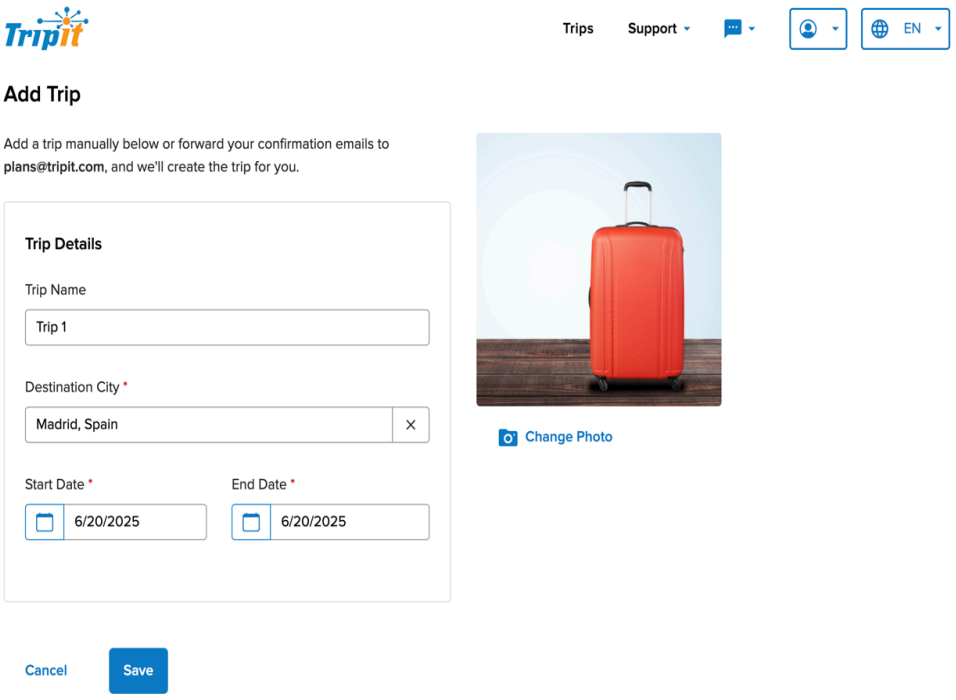
U konačnici, razvijeni sustav predstavlja suvremeno i funkcionalno rješenje koje integrira sve ključne aspekte putovanja unutar jedinstvene platforme. Fokusiran je na korisničke potrebe, ali istovremeno osigurava administrativnu kontrolu i sigurnosne mehanizme potrebne za pouzdan rad u stvarnim uvjetima.

1. ANALIZA REPREZENTATIVNIH APLIKACIJA

Suvremeno tržište nudi raznolika programska rješenja za planiranje putovanja, a svako od njih pristupa izazovu organizacije s jedinstvenim skupom alata i filozofijom.

1.1. TripIt

TripIt [1] se izdvaja svojom usmjerenošću na automatizaciju i jednostavnost. Njegova temeljna značajka leži u sposobnosti automatskog kreiranja detaljnog plana puta izravno iz potvrda rezervacija koje korisnik proslijedi putem elektroničke pošte. Ova funkcionalnost predstavlja veliku prednost jer značajno štedi vrijeme i centralizira sve ključne informacije, poput brojeva letova i adresa hotela, na jednom mjestu, dostupnom čak i bez internetske veze. Idealan je za korisnike koji cijene efikasnost i već imaju definirane elemente putovanja. Međutim, njegova snaga u organizaciji postojećih planova ujedno je i ograničenje; TripIt nudi manje podrške u fazi istraživanja novih odredišta ili aktivnosti, a neke naprednije mogućnosti, poput praćenja statusa leta u stvarnom vremenu, rezervirane su za plaćenu inačicu.

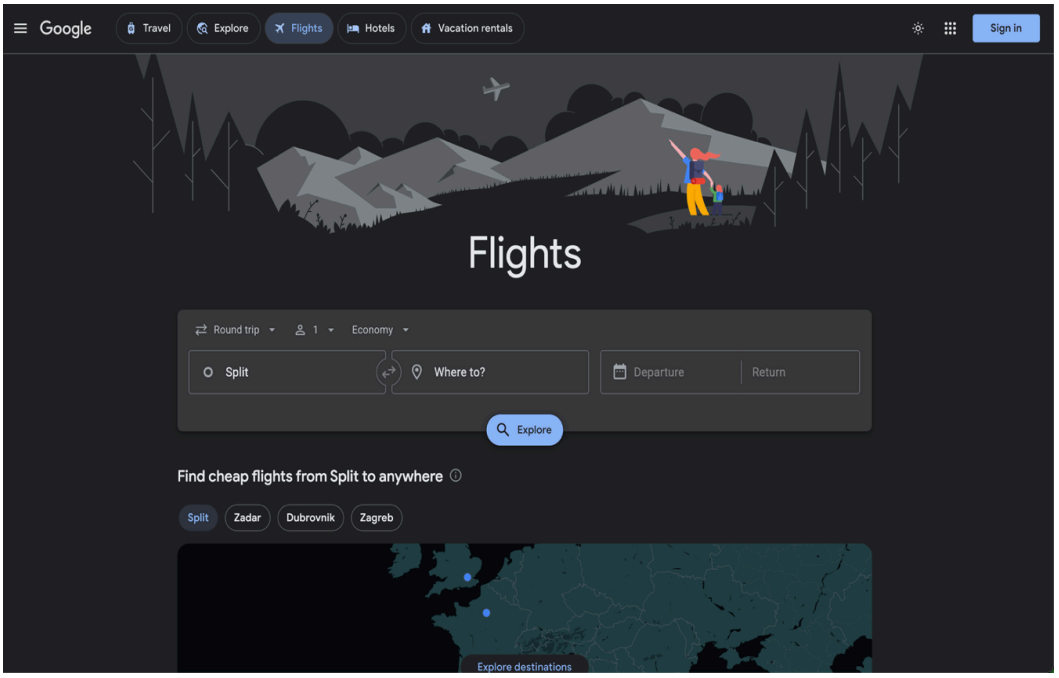


Slika 1.1 - Dodavanje putovanja u TripIt [2]

U aplikaciji je moguće dodati osnovne podatke o putovanju, a zatim ručno unositi događaje poput restorana, sastanaka i raznih aktivnosti. Itinerar se automatski organizira po danima i omogućuje jasan pregled planiranih stavki. Podaci se mogu sinkronizirati s kalendarom, a cijeli plan putovanja dostupan je i izvanmrežno. Aplikacija tako omogućuje potpunu kontrolu i preglednost svakog putovanja.

1.2. Google Travel

Google Travel [3] nudi znatno širi pristup, objedinjujući Googleove moćne alate za pretraživanje letova i hotela s mogućnostima organizacije putovanja. Njegova glavna prednost je sveobuhvatnost jer pokriva čitav spektar planiranja, od početne inspiracije i usporedbe cijena do stvaranja prilagođenih planova puta, često automatski prepoznajući rezervacije iz korisnikovog Gmail sandučića. Integracija s ostatkom Googleova ekosustava, poput Karata i Kalendara, čini korisničko iskustvo iznimno glatkim za one koji su već uronjeni u te usluge, a većina ključnih funkcionalnosti dostupna je besplatno. Ipak, upravo ta širina može ponekad djelovati opterećujuće za korisnike koji traže jednostavnije rješenje, dok pitanje privatnosti podataka ostaje relevantno, s obzirom na prirodu Googleovih usluga.



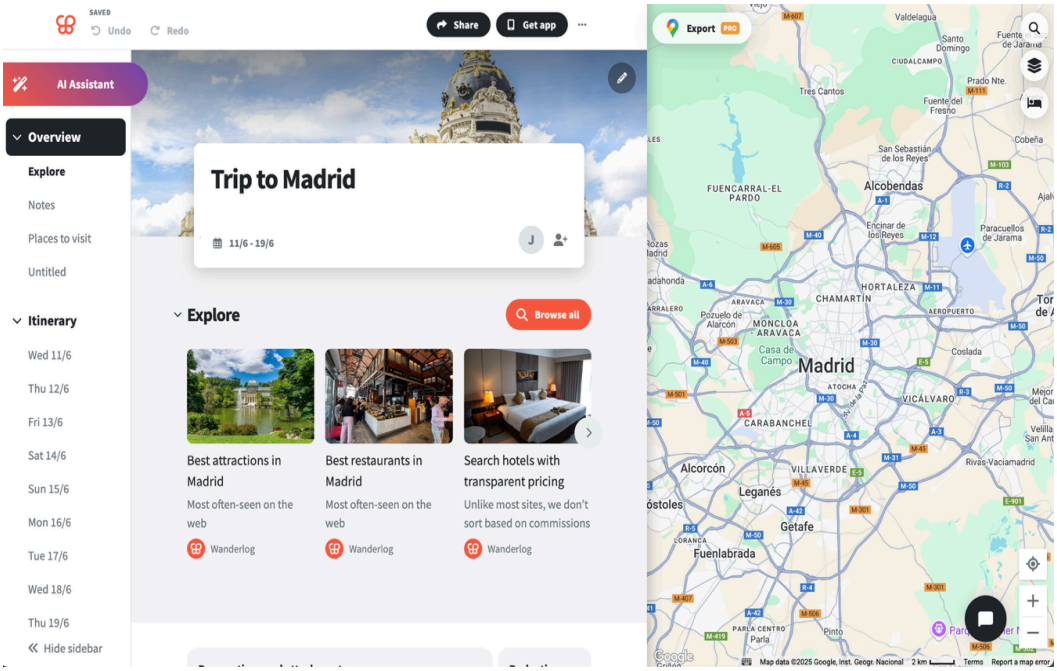
Slika 1.2 - Google Travel sučelje za letove [4]

Ovo je sučelje servisa Google Flights. Korisnik može unijeti grad polaska i odredište, odabrati datume polaska i povratka, broj putnika i klasu. Klikom na *Explore* prikazuju se dostupni letovi. Ispod se nudi pregled jeftinih letova iz odabranog grada. Isto tako funkcionira i pronalazak hotela koji su prikazani na karti grada.

1.3. Wanderlog

Za korisnike kojima je u fokusu suradničko planiranje i vizualni pristup, Wanderlog [5] se nameće kao privlačan izbor. Ova aplikacija ističe se mogućnošću da više korisnika istovremeno radi na kreiranju plana puta, dodajući mjesta i optimizirajući rute izravno na interaktivnoj karti. Takav pristup je izuzetno koristan za grupna putovanja, omogućujući transparentno i zajedničko donošenje odluka. Fleksibilnost u dodavanju detaljnih bilješki i

poveznica za svako mjesto dodatno obogaćuje proces planiranja. Iako nudi uvoz rezervacija, Wanderlog zahtijeva aktivniji angažman korisnika u usporedbi s potpunom automatizacijom TripIt. Za izradu vrlo detaljnih planova potrebno je uložiti više vremena, a sučelje, bogato opcijama, ponekad može djelovati složeno, osobito na manjim zaslonima.



Slika 1.3 - Prikaz putovanja na Wanderlog sučelju [6]

S lijeve strane sučelja nalazi se izbornik s pregledom putovanja, datumima i dnevnim itinerarom. Za svaki dan putovanja u itinerar se mogu dodati aktivnosti. U sredini su prijedlozi za atrakcije, restorane i hotele u Madridu. Desno se nalazi interaktivna karta koja prikazuje lokacije povezane s planom putovanja. Gumbi na vrhu omogućuju dijeljenje, izvoz i pristup mobilnoj aplikaciji.

2. PLAN PROJEKTA

2.1. Funkcionalni zahtjevi

1. Kreiranje planova putovanja

- Korisnik može kreirati plan putovanja kombiniranjem mrežnih podataka i vlastitog unosa.

2. Pretraga i preuzimanje podataka

- Korisnik može pretraživati i dohvatiti podatke o prijevozu, smještaju, vremenskoj prognozi, tečajnim listama, turističkim atrakcijama i ostalim točkama interesa.

3. Ručni unos i ažuriranje podataka

- Korisnik može ručno unijeti podatke koji nisu dohvaćeni putem API-ja.
- Korisnik može ažurirati podatke unutar svog plana putovanja.

4. Proračun i evidencija troškova

- Korisnik može unositi troškove vezane uz putovanje.
- Sustav izračunava ukupne troškove i prikazuje ih korisniku.

5. Prikaz lokacija na karti

- Korisniku je omogućen prikaz lokacija i atrakcija na zemljopisnoj karti.

6. Izvanmrežna pohrana planova

- Korisnik može spremiti plan putovanja za korištenje bez internetske veze.

7. Korištenje mrežnih izvora putem API-ja

- Sustav koristi dostupne API-je za dohvat podataka o prijevozu, smještaju, vremenskoj prognozi, tečajnim listama, turističkim atrakcijama i točkama interesa.

8. Upravljanje podacima
    - Korisnik može ručno ažurirati podatke o troškovima i pakiranju.
    - Korisnik može obrisati svoje putovanje.

2.2. Nefunkcionalni zahtjevi

- Odgovori na zahtjeve ne smiju trajati dulje od 10 sekundi.
- Aplikacija će biti izgrađena za Web.
- Sučelje mora biti prilagodljivo različitim veličinama zaslona.
- Aplikacija mora imati intuitivno sučelje i jednostavnu navigaciju.
- Lozinke moraju biti enkriptirane.
- Aplikacija mora podržavati izvanmrežnu pohranu korisničkih planova i podataka za korištenje bez pristupa internetu.

2.3. Ciljana skupina korisnika

Ciljana skupina korisnika obuhvaća nekoliko kategorija. Prva skupina su individualni putnici – osobe koje planiraju vlastita putovanja, uključujući solo avanturiste, kao i obitelji koje žele organizirati zajednički odmor. Druga skupina su poslovni putnici – ljudi koji često putuju zbog poslovnih obveza i kojima je potrebna brza i efikasna organizacija putovanja. Treća skupina su turističke agencije – subjekti koji koriste različite alate za brzo planiranje i upravljanje putovanjima svojih klijenata, kako bi im omogućile što bolju uslugu.

2.4. Glavni problemi korisnika

Korisnici se suočavaju s nekoliko značajnih problema prilikom planiranja putovanja. Jedan od glavnih izazova je vremenska zahtjevnost pretraživanja letova i smještaja na različitim platformama, što otežava brzo donošenje odluka. Također, postoji nedostatak centraliziranog sustava koji bi omogućio upravljanje svim aspektima putovanja na jednom mjestu, uključujući letove, smještaj, aktivnosti i posjete znamenitostima. Još jedan problem je praćenje troškova i budžeta tijekom planiranja, što je često složeno i zahtijeva dodatnu pažnju. Konačno, korisnici često nailaze na poteškoće vezane uz konverziju valuta, što može dodatno kompliciratи planiranje putovanja u inozemstvu.

2.5. Tehnološka ograničenja

Razvoj aplikacije suočava se s određenim tehnološkim ograničenjima. Prije svega, besplatni planovi API-ja poput Google Places i ExchangeRateAPI imaju ograničen broj poziva, što može utjecati na dostupnost podataka. Također, aplikacija mora biti optimizirana za performanse kako bi korisničko iskustvo bilo brzo i responzivno, što zahtijeva pažljivo upravljanje resursima. Postoji i ovisnost o stabilnosti i dostupnosti vanjskih API-ja, što može predstavljati rizik u slučaju prekida usluge ili promjena u API-ju. Osim toga, treba uzeti u obzir razlike u performansama i kompatibilnosti između različitih web preglednika kako bi aplikacija bila dostupna široj publici.

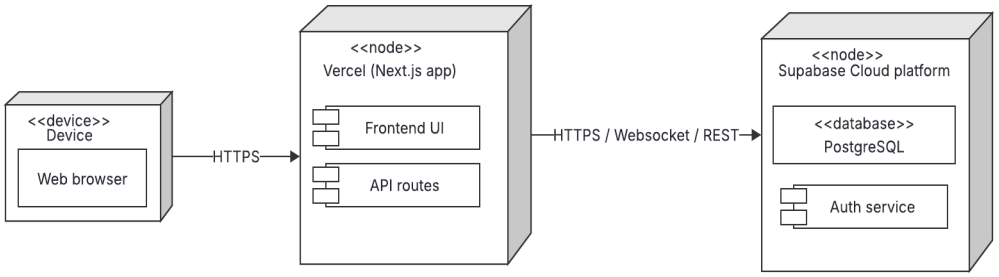
2.6. Rizici

Pri razvoju i korištenju aplikacije postoje određeni rizici koje je potrebno prepoznati i riješiti. Mogući su problemi s integracijom API-ja, naročito ako dođe do promjena u njihovoj dokumentaciji, što može zahtijevati dodatne prilagodbe. Sigurnost korisničkih podataka predstavlja važan aspekt, posebno kada se radi o osjetljivim informacijama poput lozinki. Također, promjene u uvjetima korištenja ili cijenama API-ja mogu imati značajan utjecaj na funkcionalnost aplikacije i njezine troškove, što treba uzeti u obzir prilikom planiranja dugoročnog održavanja.

3. ARHITEKTURA SUSTAVA

3.1. Osnovna arhitektura sustava

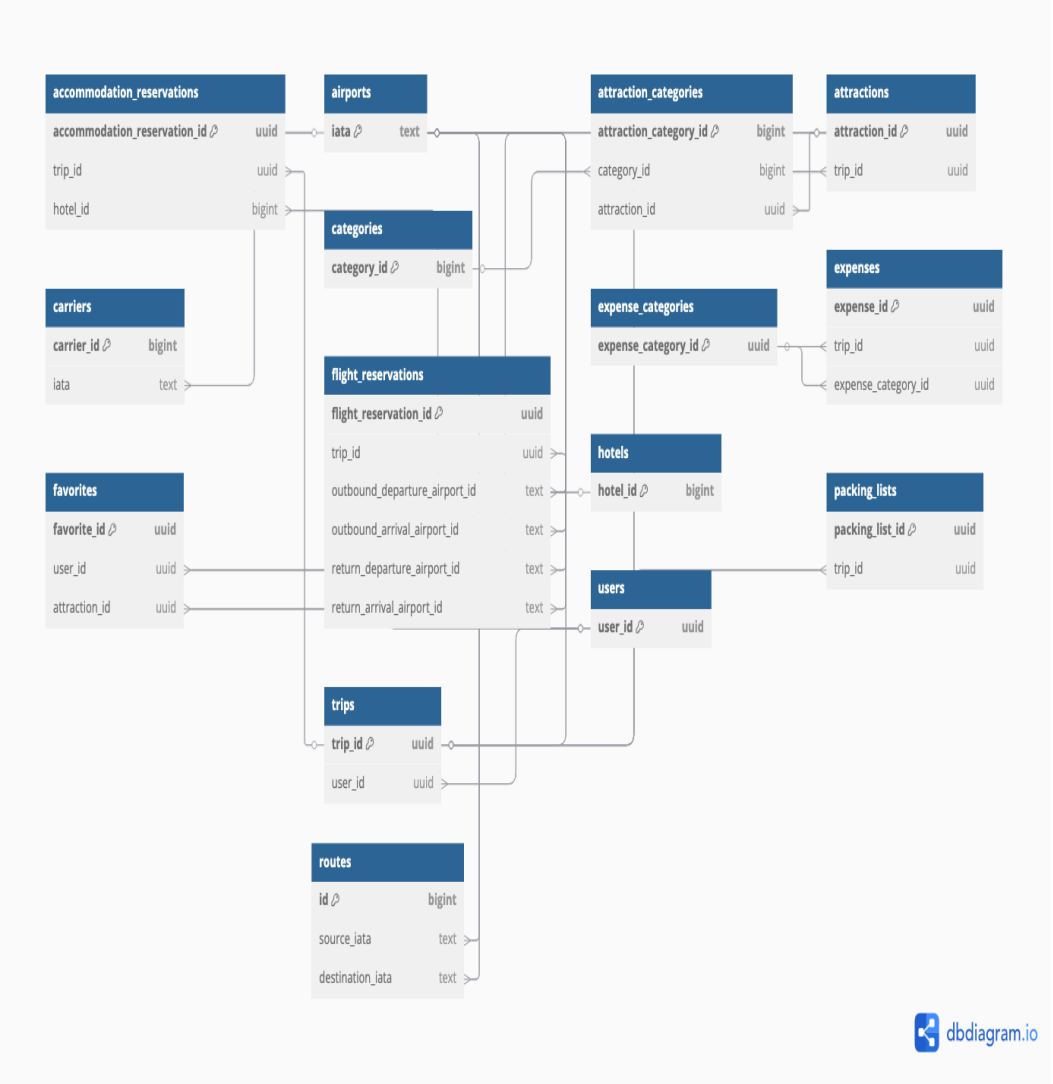
Sustav je izrađen koristeći moderni web razvojni stack koji uključuje Next.js za server-side rendering i izgradnju responzivnih korisničkih sučelja, te TypeScript za statičku tipizaciju i bolju kontrolu kvalitete koda. Za stilizaciju i dizajn komponenti koristi se Tailwind CSS, što omogućuje brzu izradu prilagodljivog i modularnog korisničkog sučelja. Podaci se pohranjuju u bazu podataka, PostgreSQL, kojom upravlja Supabase kao pozadinski (backend) servis, pružajući skalabilno i sigurno rješenje za upravljanje podacima i autentifikaciju korisnika.



Slika 3.1 Dijagram ugradnje rješenja

3.2. Baza podataka

Temelj cjelokupne aplikacije čini relacijska baza podataka, pažljivo modelirana kako bi podržala sve funkcionalnosti vezane uz planiranje i upravljanje putovanjima. Odabran je PostgreSQL kao sustav za upravljanje bazom podataka zbog svoje pouzdanosti, skalabilnosti i naprednih mogućnosti. Arhitektura baze osmišljena je s ciljem da bude fleksibilna, učinkovita i spremna za buduća proširenja, osiguravajući pritom integritet i konzistentnost podataka.



Slika 3.2 - Dijagram baze podataka s primarnim i vanjskim ključevima te vezama između tablica

Tablica 1 - Users

Naziv atributa	**Tip atributa**	**Opis atributa**
user_id (PK)	text (UUID) not null	Jedinstveni ID korisnika
first_name	text null	Ime korisnika
last_name	text null	Prezime korisnika
email	text null	Email adresa korisnika
password_hash	text not null	*Hashirana* zaporka
role	text null	Uloga korisnika
created_at	timestamp not null	Vrijeme kreiranja
updated_at	timestamp null	Vrijeme ažuriranja

Tablica *Users*, koja pohranjuje osnovne informacije o svakom registriranom korisniku, sadrži podatke poput imena, adrese e-pošte i sigurno pohranjene lozinke. Jedinstveni identifikator korisnika (*user_id*) služi kao ključna poveznica s ostalim podacima u aplikaciji, kao što su njihova putovanja ili omiljene atrakcije.

Tablica 2 - Trips

Naziv atributa	**Tip atributa**	**Opis atributa**
trip_id (PK)	text (UUID) not null	Jedinstveni ID putovanja
user_id (FK)	text (UUID) not null	Jedinstveni ID korisnika
title	text not null	Ime putovanja
destinaton	text not null	Ime grada u koji korisnik ide
start_date	date not null	Datum početka
end_date	date not null	Datum kraja
starting_budget	numeric null	Početni budžet
status	text null	Status putovanja
no_of_people	integer null	Broj ljudi na putovanju
total_cost	numeric null	Ukupan trošak putovanja
created_at	timestamp not null	Vrijeme kreiranja
updated_at	timestamp null	Vrijeme ažuriranja

Trips je tablica koja sadrži temeljne informacije o svakom planiranom putovanju, uključujući odredište, datume početka i završetka te planirani budžet. Svaki zapis u ovoj tablici povezan je s korisnikom koji ga je kreirao i predstavlja središnju točku za sve detalje vezane uz to putovanje.

Tablica 3 - Hotels

Naziv atributa	**Tip atributa**	**Opis atributa**
hotel_id (PK)	text (UUID) not null	Jedinstveni ID hotela
name	text null	Ime hotela
country_name	text null	Ime države u kojoj je hotel
city_name	text null	Ime grada
country_code	text null	Oznaka države
rating	integer null	Ocjena hotela
url	text null	URL hotela

Tablica *Hotels* pohranjuje podatke o svim hotelima dostupnima u sustavu, uključujući njihove osnovne informacije i URL web stranice. Služi kao centralni izvor informacija prilikom pretraživanja hotela u gradu koji korisnik odabere.

Tablica 4 - Airports

Naziv atributa	**Tip atributa**	**Opis atributa**
iata (PK)	text not null	Jedinstveni kod zračne luke
name	text null	Ime aerodroma
city_name	text null	Ime grada
country_name	text null	Ime države
country_code	text null	Oznaka države
timezone	text null	Vremenska zona

Tablica *Airports* sadrži popis zračnih luka s njihovim kodovima (*IATA*), nazivima i lokacijama. Koristi se za funkcionalnosti vezane uz pretraživanje i rezervaciju letova.

Tablica 5 - Carriers

Naziv atributa	**Tip atributa**	**Opis atributa**
carrier_id (PK)	integer not null	ID aviokompanije
iata (FK)	text not null	IATA aerodroma
name	text null	Ime aviokompanije

Tablica *Carriers* pohranjuje informacije o zrakoplovnim prijevoznicima, uključujući njihove nazive i kodove. Koristi se zajedno s tablicom airports za planiranje letova.

Tablica 6 - Routes

Naziv atributa	**Tip atributa**	**Opis atributa**
route_id (PK)	integer not null	ID rute
source_iata (FK)	text not null	ID polaznog aerodroma
destination_iata (FK)	text not null	ID dolaznog aerodroma
distance	integer null	Udaljenost aerodroma
duration	integer null	Trajanje leta u minutama

Tablica *Routes* definira moguće zrakoplovne rute između dviju zračnih luka, često uključujući i informacije o prijevoznicima koji prometuju na tim rutama. Pomaže u pronalaženju dostupnih letova.

Tablica 7 – Accommodation_reservations

Naziv atributa	**Tip atributa**	**Opis atributa**
accommodation_reservation_id (PK)	text (UUID) not null	ID rezervacije hotela
trip_id (FK)	text (UUID) not null	ID putovanja
hotel_id (FK)	text (UUID) not null	ID hotela
room_type	text null	Tip sobe
nights	integer null	Broj noći
price_per_night	numeric null	Cijena noći
total_price	numeric null	Ukupna cijena
currency	text null	Valuta cijene
check_in_date	date not null	Datum prijave
check_out_date	date not null	Datum odjave
created_at	timestamp not null	Vrijeme kreiranja
updated_at	timestamp null	Vrijeme ažuriranja

Accommodation_reservations pohranjuje informacije o rezerviranom smještaju tijekom putovanja. Sadrži podatke poput naziva hotela ili drugog smještajnog objekta, datume prijave i odjave, vrstu sobe i cijenu.

Tablica 8 - Flight_reservations

Naziv atributa	**Tip atributa**	**Opis atributa**
flight_reservation_id (PK)	text (UUID) not null	ID rezervacije leta
trip_id (FK)	text (UUID) not null	ID putovanja
outbound_departure_airport_id (FK)	text not null	ID polaznog aerodroma
outbound_arrival_airport_id (FK)	text not null	ID dolaznog aerodroma
return_departure_airport_id (FK)	text not null	ID polaznog aerodroma u povratku
return_arrival_airport_id (FK)	text not null	ID dolaznog aerodroma u povratku
total_price	numeric null	Ukupna cijena povratne karte
currency	text null	Valuta cijene
outbound_carrier	text null	Kratica aviokompanije prema
return_carrier	text null	Kratica aviokompanije nazad
outbound_flight_number	integer null	Broj leta prema
outbound_departure_time	timestamp null	Vrijeme polaska leta prema
outbound_arrival_time	timestamp null	Vrijeme dolaska leta prema
return_flight_number	integer null	Broj leta nazad
return_departure_time	timestamp null	Vrijeme polaska leta nazad
return_arrival_time	timestamp null	Vrijeme dolaska leta nazad
created_at	timestamp not null	Vrijeme kreiranja
updated_at	timestamp null	Vrijeme ažuriranja

Tablica *Flight_reservations* čuva detalje o rezervacijama letova za određeno putovanje. Uključuje podatke kao što su brojevi letova, zrakoplovni prijevoznici, vremena polazaka i dolazaka te cijene karata.

Tablica 9 - Packing_lists

Naziv atributa	**Tip atributa**	**Opis atributa**
packing_list_id (PK)	text (UUID) not null	ID liste pakiranja
trip_id (FK)	text (UUID) not null	ID putovanja
item_name	text null	Naziv predmeta
quantity	integer null	Količina predmeta
is_packed	boolean null	Je li spakiran predmet?
created_at	timestamp not null	Vrijeme kreiranja
updated_at	timestamp null	Vrijeme ažuriranja

Tablica *Packing_lists* omogućuje korisnicima izradu personaliziranih popisa stvari za pakiranje za svako putovanje. Svaki zapis predstavlja jednu stavku na popisu, s nazivom i statusom (spakirano/nespakirano).

Tablica 10 - Expenses

Naziv atributa	**Tip atributa**	**Opis atributa**
expense_id (PK)	text (UUID) not null	ID troška
trip_id (FK)	text (UUID) not null	ID putovanja
expense_category_id (FK)	text (UUID) not null	ID kategorije troška
description	text null	Opis troška
amount	numeric null	Trošak
currency	text null	Valuta troška
created_at	timestamp not null	Vrijeme kreiranja
updated_at	timestamp null	Vrijeme ažuriranja

Tablica *Expenses* služi za praćenje svih troškova vezanih uz određeno putovanje. Svaki redak predstavlja pojedinačni trošak s podacima o iznosu, valuti, datumu, opisu i kategoriji troška.

Tablica 11 - Expense_categories

Naziv atributa	**Tip atributa**	**Opis atributa**
expense_category_id (PK)	text (UUID) not null	ID kategorije troška
name	text not null	Ime kategorije troška
created_at	timestamp not null	Vrijeme kreiranja
updated_at	timestamp null	Vrijeme ažuriranja

Tablica *Expense_categories* definira različite vrste ili kategorije troškova (npr. Hrana, Smještaj, Prijevoz). Omogućuje dosljedan unos i lakše grupiranje troškova za analizu.

Tablica 12 - Attractions

Naziv atributa	**Tip atributa**	**Opis atributa**
attraction_id (PK)	text (UUID) not null	ID atrakcije
destination	text not null	Ime destinacije
name	text not null	Ime atrakcije
description	text null	Opis atrakcije
photo_url	text null	URL fotografije
google_place_id	text null	ID za Google Places API
types	text null	Tipovi atrakcije
rating	numeric null	Ocjena atrakcije
latitude	numeric null	Geografska širina
longitude	numeric null	Geografska dužina
location	text null	Adresa atrakcije
created_at	timestamp not null	Vrijeme kreiranja
updated_at	timestamp null	Vrijeme ažuriranja

Tablica *Attractions* sadrži katalog turističkih atrakcija s detaljnim informacijama poput naziva, opisa, lokacije i fotografija. Korisnici mogu pretraživati ove atrakcije i dodavati ih u svoje planove putovanja.

Tablica 13 - Categories

Naziv atributa	**Tip atributa**	**Opis atributa**
category_id (PK)	integer not null	ID kategorije atrakcije
name	text null	Ime kategorije atrakcije

Tablica *Categories* (za atrakcije) definira različite kategorije za turističke atrakcije (npr. Muzej, Park, Restoran). Pomaže korisnicima u filtriranju i pronalaženju atrakcija prema njihovim interesima.

Tablica 14 - Attraction_categories

Naziv atributa	**Tip atributa**	**Opis atributa**
attraction_category_id (PK)	integer not null	ID kategorije + atrakcije
category_id (FK)	integer not null	ID kategorije atrakcije
attraction_id (FK)	text (UUID) not null	ID atrakcije
created_at	timestamp not null	Vrijeme kreiranja

Tablica *Attraction_categories* je povezna tablica koja spaja atrakcije s njihovim kategorijama. Omogućuje da jedna atrakcija pripada u više različitih kategorija.

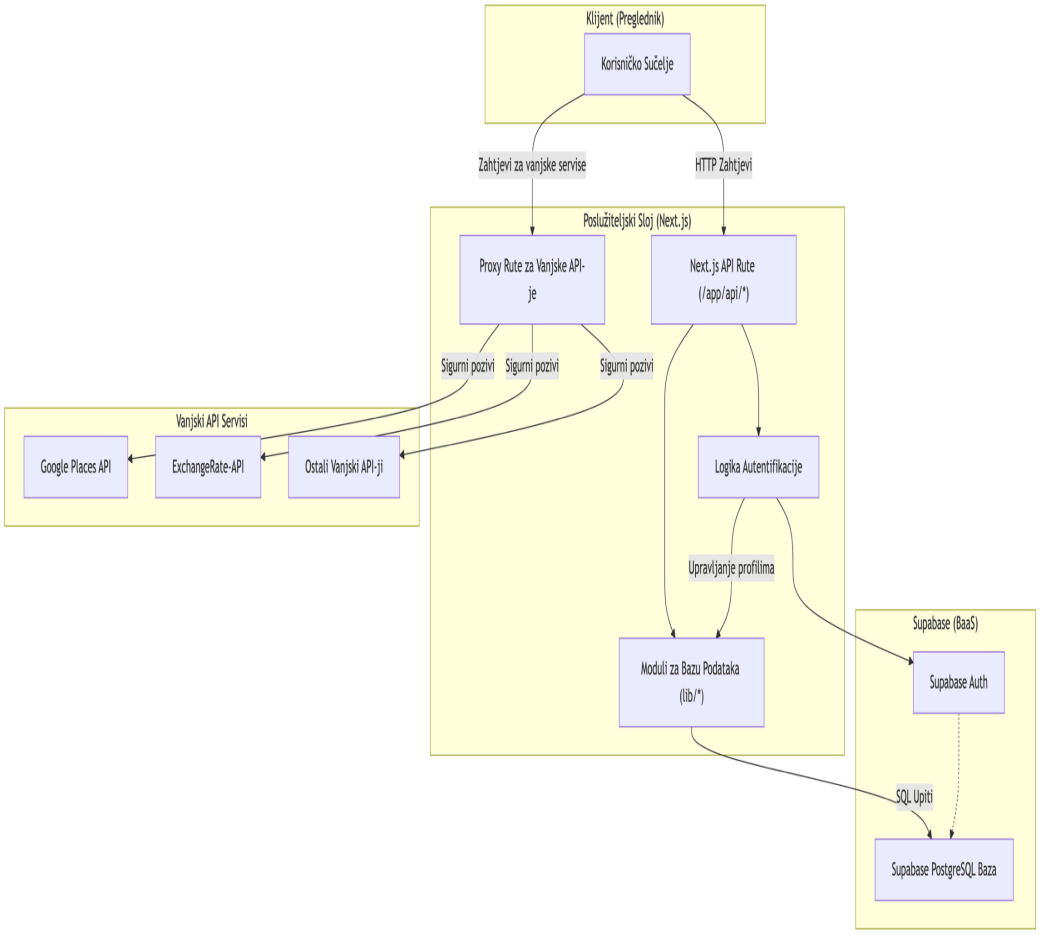
Tablica 15 - Favorites

Naziv atributa	**Tip atributa**	**Opis atributa**
favorite_id (PK)	text (UUID) not null	ID favorita
user_id (FK)	text (UUID) not null	ID korisnika
attraction_id (FK)	text (UUID) not null	ID atrakcije
created_at	timestamp not null	Vrijeme kreiranja
updated_at	timestamp null	Vrijeme ažuriranja

Tablica *Favorites* omogućuje korisnicima da spreme svoje omiljene atrakcije. Svaki zapis u ovoj tablici povezuje korisnika s određenom atrakcijom, stvarajući personalizirani popis.

Atributi *created_at* i *updated_at* koriste se za praćenje vremenskih informacija vezanih uz sve navedene tablice. *Created_at* bilježi točan trenutak kada je neki podatak prvi put unesen u sustav, dok *updated_at* označava vrijeme njegove posljednje izmjene. Ovi atributi omogućuju uvid u kronologiju aktivnosti korisnika.

3.3. Arhitektura poslužitelja



Slika 3.3 - Dijagram arhitekture poslužitelja

Aplikacija koristi Next.js kao osnovni radni okvir, što omogućuje integraciju logike sučelja (*frontend*) i pozadine (*backend*) unutar istog projekta. *Backend* je primarno implementiran putem *Serverless* funkcija koje se izvršavaju kao Next.js API rute. Za pohranu podataka, autentifikaciju i dodatne backend usluge koristi se platforma za razvoj Supabase. Ključne prednosti ove arhitekture uključuju skalabilnost, budući da se *Serverless* funkcije automatski prilagođavaju potražnji, te održivost zahvaljujući jasnoj podjeli logike u modulima (lib/) i API rutama. Razvoj je znatno ubrzan korištenjem Next.js-a i Supabasea, dok sigurnost osiguravaju robusni mehanizmi Supabase platforme te činjenica da su osjetljivi API ključevi i poslovna logika smješteni na poslužiteljskoj strani. Performanse su dodatno poboljšane mogućnošću korištenja *Edge* funkcija i različitih strategija predmemoriranja (*caching*).

Ključne komponente arhitekture uključuju:

1. Next.js API Rute (*Route Handlers*): Smještene u direktoriju *app/api/*, ove datoteke definiraju backend logiku. Svaka ruta obrađuje specifične HTTP zahtjeve (*GET, POST, DELETE*, itd.) za određene funkcionalnosti.

2. Servisni moduli (u direktoriju lib/):

- *lib/database.ts* i *lib/attractions-db.ts*: Sadrže funkcije za interakciju sa Supabase bazom podataka (čitanje, pisanje, ažuriranje, brisanje podataka iz tablica kao što su *trips, airports, attractions, users* itd.).
- *lib/supabase.ts*: Konfigurira Supabase klijent i pruža osnovne funkcije za autentifikaciju i sesije.
- *lib/auth.ts*: Sadrži napredniju logiku za autentifikaciju korisnika (registracija, prijava, odjava) koja se nadograđuje na Supabase Auth i koristi prilagođenu *users* tablicu.

3. Supabase:

- Baza podataka: PostgreSQL baza podataka kojom se upravlja putem Supabase sučelja i klijentskih biblioteka. Shema baze podataka definirana je SQL skriptom u direktoriju database/.
- Autentifikacija: Supabase Auth se koristi za upravljanje korisničkim identitetima, sesijama i sigurnošću. Aplikacija proširuje ovo s prilagođenom users tablicom za dodatne korisničke informacije.

4. Vanjske API integracije – aplikacija komunicira s vanjskim servisima kao što su:

- Google Places API: Za dohvaćanje informacija o atrakcijama, fotografijama i mjestima. Pozivi prema ovom API-ju su proslijeđeni kroz vlastite API rute (*app/api/attractions/google-photo/, app/api/attractions/google-places/*).
- ExchangeRate-API: Za dohvaćanje tečajeva valuta, također proxyrano (*app/api/exchange-rates/*).

U nastavku su detaljnije objašnjene pojedine komponente poslužitelja.

3.3.1. Next.js API rute

Lokacija: *app/api/**/route.ts*

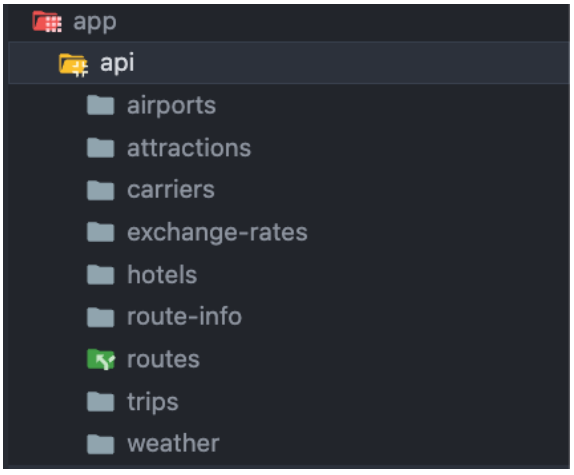
Funkcionalnost: Ove datoteke djeluju kao *backend endpointi*. Primaju HTTP zahtjeve s klijentskog dijela aplikacije, obrađuju ih (primjerice, validacija ulaznih podataka, pozivanje servisnih modula za pristup bazi podataka ili vanjskim API-jima) i vraćaju HTTP odgovore – obično u JSON formatu koristeći *NextResponse*.

Primjeri API ruta:

- *app/api/airports/route.ts*: Pretraživanje zračnih luka, dohvaćanje polaznih i odredišnih gradova.
- *app/api/attractions/route.ts*: Dohvaćanje atrakcija, filtriranje po kategorijama, uklanjanje duplikata.
- *app/api/attractions/import/route.ts*: Uvoz atrakcija iz Google Places API-ja u bazu podataka.
- *app/api/trips/[id]/attractions/route.ts*: Upravljanje atrakcijama vezanim za određeno putovanje.
- *app/api/weather/route.ts*: Trenutno vraća *mock*[1] podatke o vremenu, ali je predviđeno povezivanje s pravim vremenskim API-jem.

Caching: Neke rute koriste HTTP cache zaglavlja poput *Cache-Control*, s-maxage i stale-while-revalidate radi optimizacije performansi i smanjenja opterećenja na vanjske servise.

Runtime: Određene rute koriste *export const runtime = "edge";*, čime se omogućuje njihovo izvođenje na Vercel Edge mreži radi što manje latencije i bržeg odgovora korisniku.



Slika 3.4 - Struktura direktorija app/api

3.3.2. Interakcija s bazom podataka

Moduli zaduženi za interakciju s bazom podataka, poput *lib/database.ts* i *lib/attractions-db.ts*, djeluju kao apstrakcijski sloj između aplikacijske logike i Supabase baze podataka. Oni pojednostavljuju izravnu komunikaciju koristeći službenog Supabase JavaScript klijenta (*@supabase/supabase-js*) za izvršavanje SQL operacija kao što su *SELECT, INSERT, UPDATE* i *DELETE*. U modulu je također uključena logika za obradu grešaka, s detaljnim zapisivanjem grešaka pomoću funkcije *logDetailedError*, što olakšava dijagnostiku problema tijekom izvođenja API poziva.

Unutar ovih modula nalaze se funkcije specifične za domenu aplikacije, kao što su *getHotelsByCity()* za dohvaćanje hotela, *createTrip()* za stvaranje novog putovanja, *getAttractionsByCity()* za dohvaćanje atrakcija ili *addFavorite()* za dodavanje atrakcije u listu favorita korisnika.

Posebna pažnja posvećena je upravljanju korisničkim podacima. Iako se osnovna autentifikacija (registracija, prijava) oslanja na Supabase Auth, aplikacija koristi i prilagođenu tablicu *users* za pohranu dodatnih informacija o korisnicima (npr. ime, prezime). Modul *lib/auth.ts* sadrži logiku za interakciju s ovom tablicom.

Pohranjivanje i ažuriranje korisničkih podataka odvija se na sljedeći način:

- Pohranjivanje novog korisnika: Nakon što Supabase Auth uspješno registrira novog korisnika, aplikacijska logika poziva funkciju koja umeće zapis u tablicu *users*. U taj zapis se pohranjuju dodatni korisnički podaci, a *user_id* dobiven od Supabase Auth koristi se za povezivanje. Povučeni podaci iz forme za registraciju (npr. ime, prezime, email) direktno se umeću u odgovarajuće stupce tablice *users*.
- Ažuriranje korisničkih podataka: Kada korisnik želi ažurirati svoj profil (npr. promijeniti ime), klijentski dio aplikacije šalje zahtjev API ruti zaduženoj za ažuriranje profila. Ta API ruta zatim poziva funkciju koja izvršava *UPDATE* SQL operaciju na tablici *users*, mijenjajući samo one podatke koje je korisnik specificirao. Podaci za ažuriranje dolaze iz forme koju je korisnik ispunio na klijentskoj strani.

3.3.3. Autentifikacija

Aplikacija koristi Supabase Auth kao osnovni mehanizam za registraciju, prijavu i upravljanje korisničkim sesijama.

Modul *lib/supabase.ts* inicijalizira Supabase klijent i pruža osnovne funkcije za rad sa sesijama:

- *getSession()* – dohvaća trenutnu sesiju,
- *getCurrentUser()* – dohvaća trenutno prijavljenog korisnika,
- *signInAnonymously()* – omogućuje anonimnu prijavu (npr. za testiranje),
- *DEFAULT_USER_ID* – koristi se kada korisnik nije autentificiran.

Modul lib/auth.ts proširuje Supabase autentifikaciju interakcijom s prilagođenom tablicom *users*:

- Funkcije *signUp(), signIn(), signOut()* upravljaju ciklusom korisničke sesije.

- Prilikom registracije, korisnik se stvara i u Supabase Auth i u users tablici, gdje se pohranjuju dodatni podaci kao što su ime, prezime i uloga.
- Prilikom prijave, korisnik se autentificira putem Supabase Auth, a zatim se dohvaćaju dodatni podaci iz users tablice.
- *simpleHash* funkcija koristi se za hashiranje lozinki u demonstracijske svrhe (u produkciji se koristi Supabaseova interna zaštita lozinki).

3.3.4. Vanjske API integracije

Aplikacija ne komunicira izravno s vanjskim API servisima putem klijenta (korisničkog preglednika), već koristi vlastite Next.js API rute kao posrednički sloj. Ovakva arhitektura donosi više ključnih prednosti:

- Sigurnost: Osjetljivi API ključevi (npr. *GOOGLE_API*, *EXCHANGE_RATE_KEY*) pohranjeni su na poslužitelju kao varijable okruženja i nikada nisu izloženi u kodu koji se izvršava u pregledniku.
- *Caching* (Predmemoriranje): Poslužitelj može implementirati mehanizme za predmemoriranje odgovora vanjskih API-ja. Time se smanjuje broj poziva prema vanjskim servisima, što rezultira manjim troškovima i bržim odzivom aplikacije za česte upite.
- Transformacija podataka: Odgovori vanjskih servisa mogu se filtrirati, prilagoditi ili kombinirati prije nego što se pošalju klijentu, čime se smanjuje količina podataka koja se prenosi preko mreže i pojednostavljuje obrada na klijentskoj strani.

Integrirani API servisi uključuju Google Places API koji se koristi za dohvaćanje i obogaćivanje podataka o atrakcijama i ExchangeRate-API koji se koristi za dohvaćanje valutnih tečajeva i konverziju valuta.

U nastavku je prikazano kako funkcionira ova integracija kroz primjere koda.

Kada korisnik želi vidjeti prijedloge atrakcija, primjerice pri učitavanju stranice ili nakon unosa pojma za pretragu, klijentska komponenta (*components/attractions-tab.tsx*) ne poziva Googleov servis izravno. Umjesto toga, šalje *fetch* zahtjev na internu API rutu */api/attractions/google-places*.

```
// Unutar funkcije fetchSuggestedAttractions:
// ...
  const params = new URLSearchParams({ query: `turističke atrakcije u ${desti
  const googleUrl = `/api/attractions/google-places?${params.toString()}`;
  const response = await fetch(googleUrl);
  const data = await response.json();
  // ... obrada podataka ...
// ...
```

Slika 3.5 - Slanje zahtjeva za atrakcije na internu API rutu

Ovaj zahtjev zatim obrađuje poslužiteljska ruta (*app/api/attractions/google-places/route.ts)*. Ona prima upit od klijenta, dohvaća tajni ključ *GOOGLE_API* iz varijabli okruženja, sastavlja puni URL za stvarni Google Places API, šalje zahtjev prema Googleovom servisu, te prima i prosljeđuje odgovor natrag klijentu.

```
import { NextResponse } from "next/server";

export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const query = searchParams.get("query");
  const apiKey = process.env.GOOGLE_API; // Dohvaćanje tajnog ključa

  if (!apiKey || !query) { /* ... obrada greške ... */ }

  const url = `https://maps.googleapis.com/maps/api/place/textsearch/json?que

  try {
    const response = await fetch(url, { next: { revalidate: 3600 } }); // Cac
    const data = await response.json();
    return NextResponse.json(data);
  } catch (error) { /* ... obrada greške ... */ }
}
```

Slika 3.6 - Dohvaćanje tajne, slanje zahtjeva i primanje odgovora za Google Places API

Dohvaćanje tečajeva valuta (ExchangeRateAPI):

Sličan princip primjenjuje se i za dohvaćanje tečajeva valuta. Kada korisnik želi izvršiti konverziju, klijentska strana (npr. putem uslužne funkcije u *utils/currency-utils.ts*) poziva internu API rutu */api/exchange-rates*.

```
// Unutar funkcije getExchangeRates:
// ...
  const response = await fetch(`/api/exchange-rates?base=${baseCurrency}`);
  const data = await response.json();
  // ... obrada podataka ...
// ...
```

Slika 3.7 - Slanje zahtjeva za tečaj valute na internu API rutu

Poslužiteljska ruta (*app/api/exchange-rates/route.ts*) zatim preuzima zahtjev, koristi svoj tajni API ključ za komunikaciju s ExchangeRateAPI servisom i vraća dobivene tečajeve klijentu.

```
import { NextResponse } from "next/server";

const API_KEY = process.env.EXCHANGE_RATE_KEY; // Dohvaćanje tajnog ključa
const BASE_URL = "https://v6.exchangerate-api.com/v6";

export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const base = searchParams.get("base") || "EUR";

  if (!API_KEY) { /* ... obrada greške ... */ }

  try {
    const response = await fetch(`${BASE_URL}/${API_KEY}/latest/${base}`);
    const data = await response.json();
    return NextResponse.json(data);
  } catch (error) { /* ... obrada greške ... */ }
}
```

Slika 3.8 - Obrađivanje zahtjeva i odgovora u internoj API ruti

Ovaj posrednički pristup osigurava da je aplikacija sigurnija, jer osjetljivi ključevi ostaju na poslužitelju, te potencijalno brža zahvaljujući mogućnostima predmemoriranja na poslužiteljskoj strani.

3.3.5. Okruženje i postavljanje

Aplikacija koristi *environment* varijable za konfiguraciju osjetljivih podataka, uključujući:

- Supabase URL i javne/privatne ključeve,
- Google API ključ,
- Ključeve za druge servise (npr. Exchange Rate API).

Aplikacija je optimizirana za deployment na platformi poput Vercela, koja *nativno* podržava Next.js i izvršavanje serverless funkcija (*Edge Functions*). Time se omogućuje visoka dostupnost i niska latencija diljem globalne mreže.

3.4. Arhitektura korisničkog sučelja

Korisničko sučelje aplikacije pruža *Single Page Application* (SPA) iskustvo, dok Next.js *App Router* omogućuje i serversko renderiranje te statičku generaciju radi optimalnih performansi. Arhitektura je modularna, s jasnom podjelom odgovornosti između različitih dijelova sučelja, što omogućuje da komponente budu neovisne i lako ponovno koriste. Next.js *App Router* nameće organiziranu strukturu projekta, a performanse se dodatno poboljšavaju korištenjem serverskog renderiranja i dostupnih optimizacija. Razvojni proces je pojednostavljen zahvaljujući alatima kao što su TypeScript, Tailwind CSS i React Context. TypeScript dodatno povećava sigurnost tipova, smanjujući broj grešaka i olakšavajući razumijevanje koda.

Ključne komponente arhitekture korisničkog sučelja su:

- Next.js App Router: Osnova za navigaciju i strukturu stranica. Koristi se datotečni sustav za definiranje ruta.
- React Komponente: Gradivni blokovi sučelja, organizirani u stranice i ponovno iskoristive komponente.
- *State Management* (Upravljanje Stanjem): Kombinacija lokalnog stanja komponenti, React Context API-ja i localStorage za perzistenciju.
- *Styling*: Primarno Tailwind CSS za brz i responzivan dizajn.
- *Data Fetching* (Dohvaćanje Podataka): Klijentsko dohvaćanje podataka iz Next.js API ruta.
- Tipovi i Uslužne Funkcije: TypeScript za sigurnost tipova i uslužne funkcije za pomoćne operacije.

3.4.1. Next.js App Router i Navigacija

Struktura ruta: Koristi se *app/* direktorij. Svaki folder unutar *app/* koji sadrži *page.tsx* datoteku definira javnu rutu.

- Primjer: *app/trips/new/page.tsx* odgovara ruti */trips/new*.

Dinamičke rute: Koriste se za stranice koje prikazuju specifične entitete.

- Primjer: *app/trips/[id]/page.tsx* za prikaz detalja pojedinog putovanja (npr. */trips/123*)
- Primjer: app/explore/[city]/page.tsx za istraživanje atrakcija u određenom gradu.

Layout komponente (*app/layout.tsx*): Definira osnovni HTML kostur aplikacije, uključujući globalne stilove, meta tagove i omotače za kontekst (npr. *AuthProvider, TripProvider*). Sadrži i glavnu navigaciju (*MainNav*).



Slika 3.9 - Prikaz omotača za kontekst i glavnu navigaciju u layout.tsx

Navigacija:

- *next/link* komponenta za deklarativnu navigaciju između stranica
- *next/navigation* hook (*useRouter*) za programatsku navigaciju (npr. nakon uspješne prijave ili kreiranja putovanja).

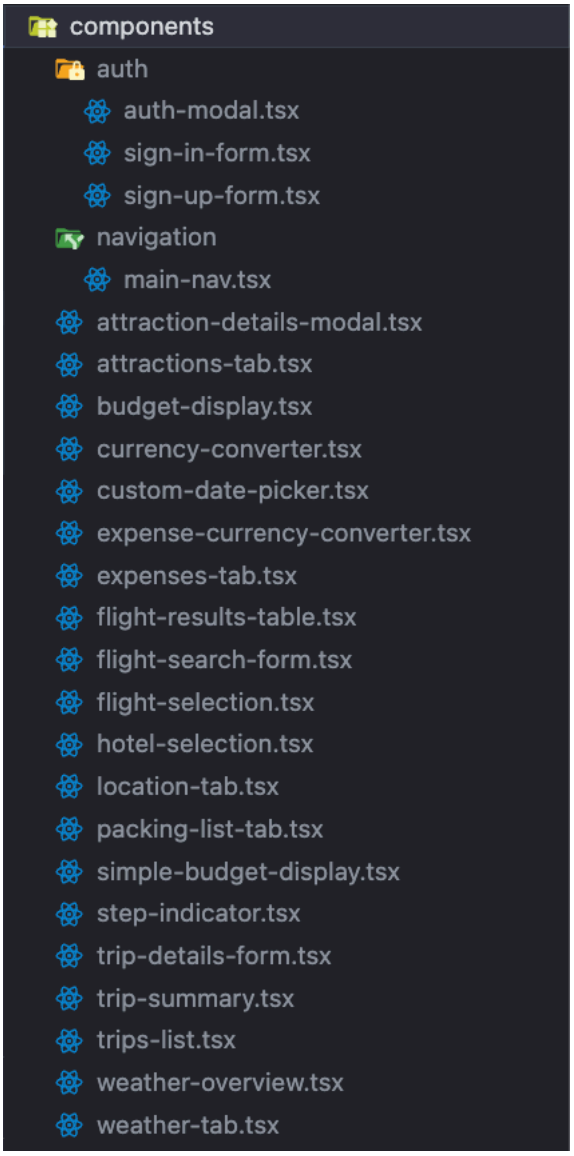
3.4.2. React komponente

Stranice (app/**/page.tsx):

- Predstavljaju glavne prikaze za svaku rutu.
- Često su označene s *use client* jer upravljaju stanjem, dohvaćaju podatke na klijentu ili koriste React hookove.
- Sastavljaju korisničko sučelje koristeći manje, ponovno iskoristive komponente.
- Primjer: *app/trips/[id]/page.tsx* dohvaća podatke o specifičnom putovanju i prikazuje ih kroz tabove (*Overview, Location, Expenses* itd.).

Ponovno iskoristive komponente (*components/*):

- UI Elementi: iz *shadcn/ui* biblioteke (npr. *Button, Card, Input, Select*)
- Prilagođene UI komponente
- Funkcionalne komponente
- *Layout* komponente.



Slika 3.10 - Direktorij ponovno iskoristivih komponenti

3.4.3. Upravljanje stanjem

Lokalno stanje komponenti (*useState, useRef*):

- Koristi se unutar pojedinačnih komponenti za upravljanje UI stanjem, kao što su vrijednosti formi, vidljivost modala, stanja učitavanja, itd.

```tsx
export default function TripDetailsForm({ onSubmit, initialData }: TripDetailsFormProps) {
  const [formData, setFormData] = useState({
    title: initialData.title || "",
    originCity: initialData.originCity || "",
    destinationCity: initialData.destinationCity || "",
    startDate: initialData.startDate || "",
    endDate: initialData.endDate || "",
    numTravelers: initialData.numTravelers || 1,
    budget: initialData.budget || 1000,
  })

  const [allCities, setAllCities] = useState<string[]>([])
  const [originCities, setOriginCities] = useState<string[]>([])
  const [destinationCities, setDestinationCities] = useState<string[]>([])
  const [isLoadingOrigin, setIsLoadingOrigin] = useState(false)
  const [isLoadingDestination, setIsLoadingDestination] = useState(false)
  const [showOriginDropdown, setShowOriginDropdown] = useState(false)
  const [showDestinationDropdown, setShowDestinationDropdown] = useState(false)
  const [originSearch, setOriginSearch] = useState("")
  const [destinationSearch, setDestinationSearch] = useState("")
  const [error, setError] = useState<string | null>(null)
  const [isFiltered, setIsFiltered] = useState(false)
```

Slika 3.11 - Primjer upravljanja stanjem u components/trip-details-form.tsx

React context API (*contexts/*):

- *contexts/auth-context.tsx*:
  - Pruža globalno stanje autentifikacije (trenutni korisnik, je li autentificiran, status učitavanja).
  - Nudi funkcije za prijavu (*signIn*), registraciju (*signUp*) i odjavu (*signOut*).
  - Koristi *useAuth* hook za pristup kontekstu unutar komponenti
- *contexts/trip-context.tsx*:
  - Upravlja stanjem vezanim za putovanja (dohvaćanje svih putovanja, dohvaćanje pojedinog putovanja, trenutno odabrano putovanje, kreiranje/brisanje putovanja).
  - Koristi *useTrip* hook.

LocalStorage:

- Koristi se za perzistenciju nekih podataka na strani klijenta kako bi se poboljšalo korisničko iskustvo.
- Primjer u *app/trips/new/page.tsx*: Spremanje napretka forme za novo putovanje (*nomad_trip_data*).
- Primjer u *components/weather-tab.tsx*: Keširanje podataka o vremenu (*weather_${tripId}*).

URL kao izvor stanja:

- Parametri rute (npr. *tripId u /trips/[id]*) se koriste za dohvaćanje specifičnih podataka i upravljanje prikazom.

3.4.4. Stiliziranje

- Tailwind CSS: Glavni alat za stiliziranje. Klase poput *bg-teal-500, text-white, grid, md:grid-cols-2, p-4, rounded-lg* su vidljive kroz cijeli projekt.
- Globalni Stilovi (*app/styles/globals.css*): Sadrži osnovne Tailwind direktive (*@tailwind base*; *@tailwind components*; *@tailwind utilities*;) i eventualne globalne prilagodbe.
- Responzivni Dizajn: Postignut korištenjem Tailwind-ovih responzivnih prefiksa (npr. *md:, lg:*).
- Dinamičko Stiliziranje: Uvjetno dodavanje klasa pomoću JavaScript logike (npr. za aktivne kartice, odabrane elemente).

```
{hotels.map((hotel) => (
  <div
    key={hotel.hotel_id}
    className={`border rounded-lg overflow-hidden hover:shadow-md transition ${
      selectedHotel?.hotel_id === hotel.hotel_id ? "border-teal-500 ring-1 ring-teal-500" : "border-gray-200"
    }`}
  >
```

Slika 3.12 - Primjer dinamičkog stiliziranja

3.4.5. Dohvaćanje podataka

Klijentsko dohvaćanje:

- Unutar komponenti označenih s *use client*, *useEffect* hook se često koristi za dohvaćanje podataka nakon što se komponenta montira.
- Koristi se globalni fetch API za slanje HTTP zahtjeva prema Next.js API rutama (npr. */api/airports*, */api/hotels*).
- Komponente upravljaju stanjima učitavanja (*isLoading*) i grešaka (*error*) kako bi pružile povratnu informaciju korisniku.

```
useEffect(() => {
  const fetchAirportsAndFlights = async () => {
    setIsLoading(true);
    setError(null);
    try {
      // ... logika za dohvaćanje aerodroma i letova ...
      const originResponse = await fetch(`/api/airports?keyword=${enco
      // ...
    } catch (err) {
      // ... obrada greške ...
    } finally {
      setIsLoading(false);
    }
  };
  fetchAirportsAndFlights();
}, [originCity, destinationCity, /* ... */]);
```

Slika 3.13 - Primjer u components/flight-selection.tsx

Serversko dohvaćanje:

Stranice u App Routeru mogu dohvaćati podatke na serveru prije renderiranja. app/trips/[id]/page.tsx parametre rute raspakirava React funkcijom use(), a podatke zatim dohvaća funkcijom fetchTripById iz konteksta.

3.4.6. Tipovi i uslužne funkcije

TypeScript (*types/*):

- Definirana su sučelja (*interfaces*) za sve ključne entitete aplikacije (npr. Trip, Flight, Hotel, Attraction, User, Expense). Ovo osigurava sigurnost tipova i bolju čitljivost koda.
- Primjer: *types/trip.ts, types/flight.ts*.

Uslužne Funkcije (*utils/*):

- Sadrže pomoćne funkcije koje se koriste na više mjesta u aplikaciji.
- *utils/date-utils.ts*: Funkcije za formatiranje i parsiranje datuma.
- *utils/currency-utils.ts*: Funkcije za rad s valutama.
- *utils/airports.ts*: Sadrži funkciju za dohvaćanje i privremeno spremanje podataka o zračnim lukama.

3.4.7. Korisničko iskustvo (UX) i UI obrasci

Modali:

- *components/auth/auth-modal.tsx*: Za prijavu i registraciju korisnika.

- *components/attraction-details-modal.tsx*: Za prikaz detalja atrakcije s mapom.

Tabovi:

- Unutar stranice s detaljima putovanja (*app/trips/[id]/page.tsx*), koriste se tabovi za organizaciju različitih aspekata putovanja (pregled, lokacija, troškovi, pakiranje, atrakcije, vrijeme).

Indikatori Koraka:

- *components/step-indicator.tsx*: Koristi se u procesu kreiranja novog putovanja (*app/trips/new/page.tsx*) za vođenje korisnika kroz faze.

Povratne Informacije Korisniku:

- Ikone za učitavanje (*Loader2* iz *lucide-react*) se prikazuju tijekom asinkronih operacija.
- Poruke o greškama se prikazuju korisniku.
- Potvrdne poruke ili vizualne promjene nakon uspješnih akcija.

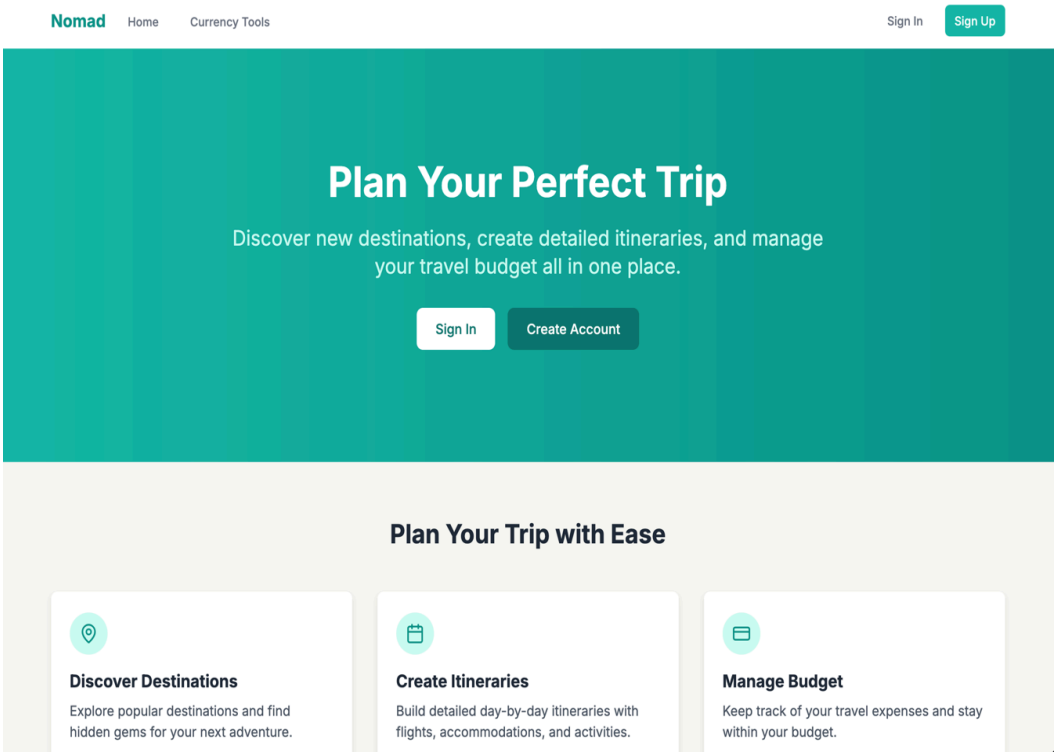
Google Maps Integracija:

- *components/location-tab.tsx* i *components/attraction-details-modal.tsx* koriste Google Maps API za prikazivanje interaktivnih mapa, markera i detalja o mjestima. Skripta za Google Maps se dinamički učitava.

4. FUNKCIONALNOSTI APLIKACIJE

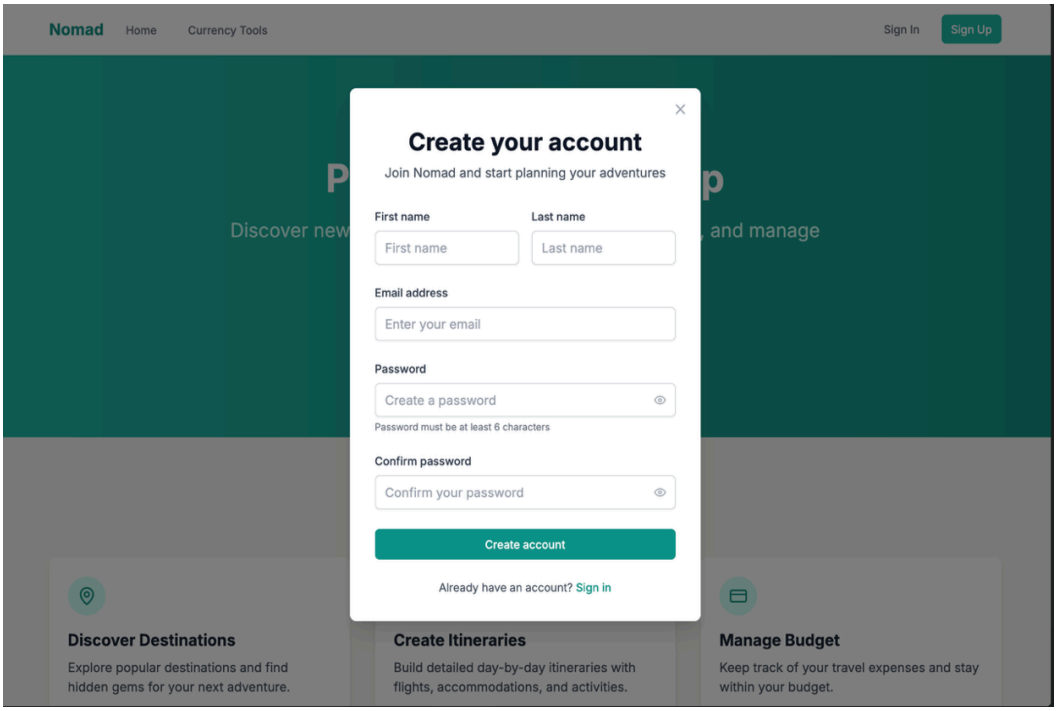
4.1. Autentifikacija

Ovo je prvi ekran koji se prikazuje kada korisnik pristupi web stranici. U navigacijskoj traci ili na glavnom ekranu korisnik se može registrirati ili prijaviti ako već ima korisnički račun.



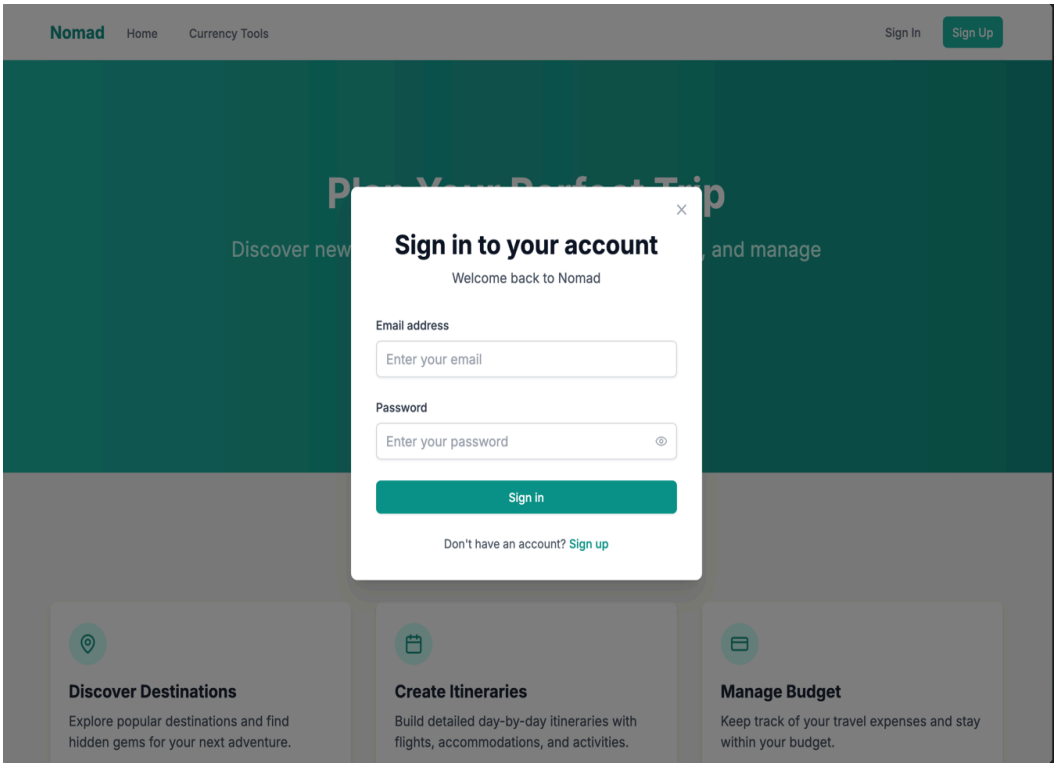
Slika 4.1 – Početni ekran anonimnog korisnika

Nakon klika na gumb *Create account* na glavnom dijelu stranice ili gumb *Sign up* u navigacijskoj traci, prikazuje se forma za registraciju koja sadrži polja za unos imena, prezimena, e-mail adrese i zaporke korisnika.



Slika 4.2 - Forma za registraciju

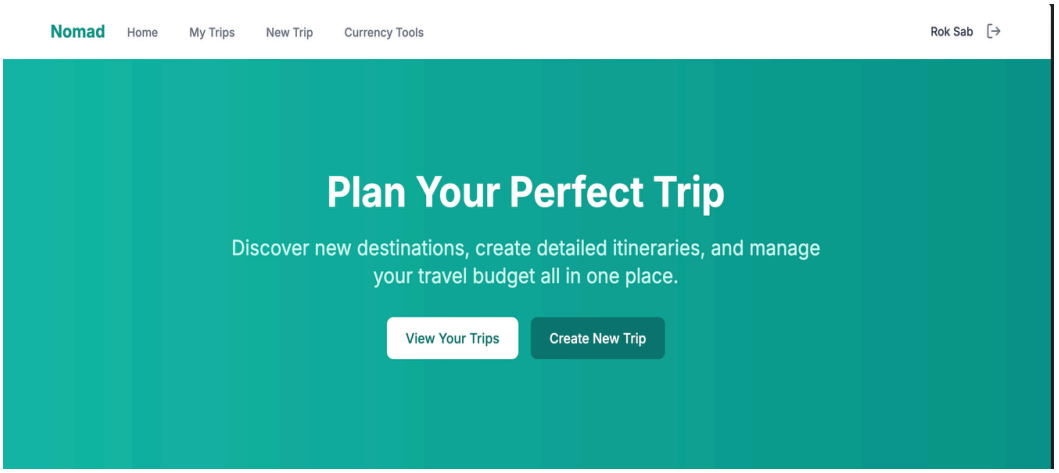
Nakon klika na gumb *Sign in* prikazuje se forma za prijavu koja sadrži polja za unos e-mail adrese i zaporke korisnika.



Slika 4.3 - Forma za prijavu

Nakon prijave, odnosno registracije, glavni zaslon stranice mijenja se; korisnik vidi nove opcije u navigacijskoj traci i glavni zaslon ima nove gumbe.

Korisnik se uvijek može vratiti na početnu stranicu pritiskom na *Home* u navigacijskoj traci.

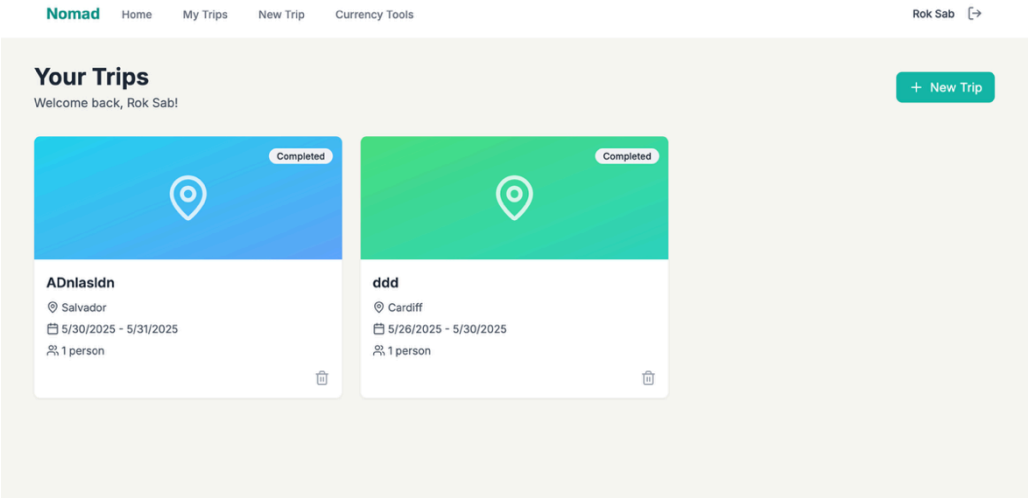


Slika 4.4 - Početna stranica za prijavljenog korisnika

Korisnik se također uvijek može odjaviti pritiskom na ikonu pokraj svog imena, na desnoj strani navigacijske trake.

4.2. Putovanja

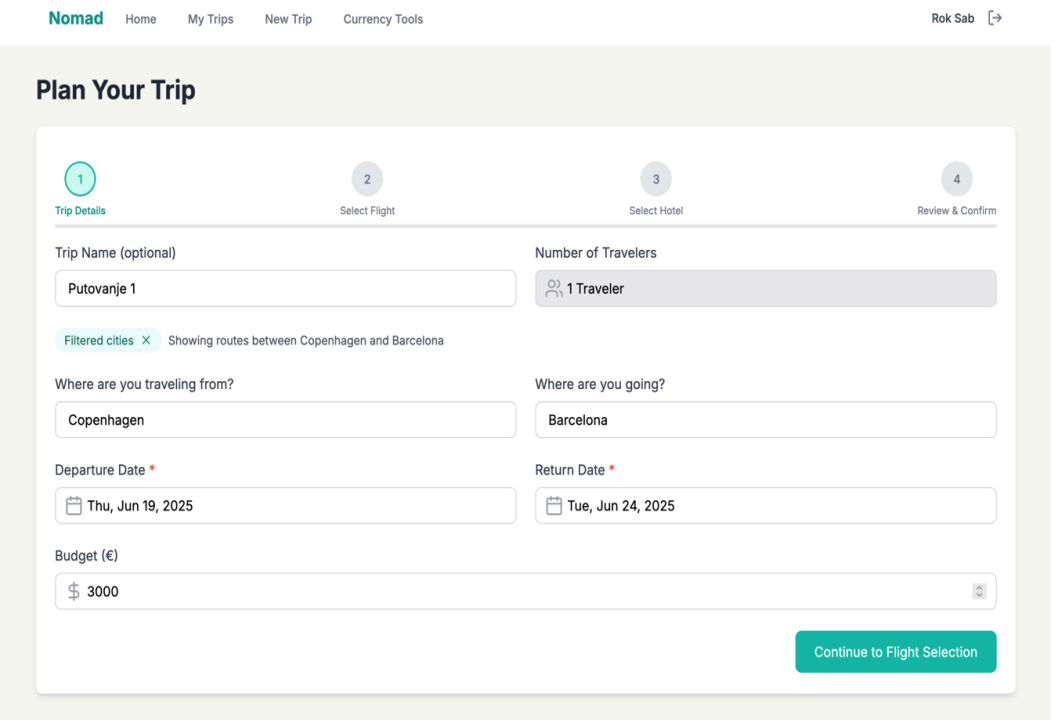
Pritiskom na *My trips* u navigacijskoj traci ili *View your trips* na početnom zaslonu, korisnik dobiva pregled svih svojih putovanja. Pojedinačno putovanje se može obrisati klikom na ikonicu kante za otpad.



Slika 4.5 - Pregled putovanja

4.2.1. Stvaranje putovanja

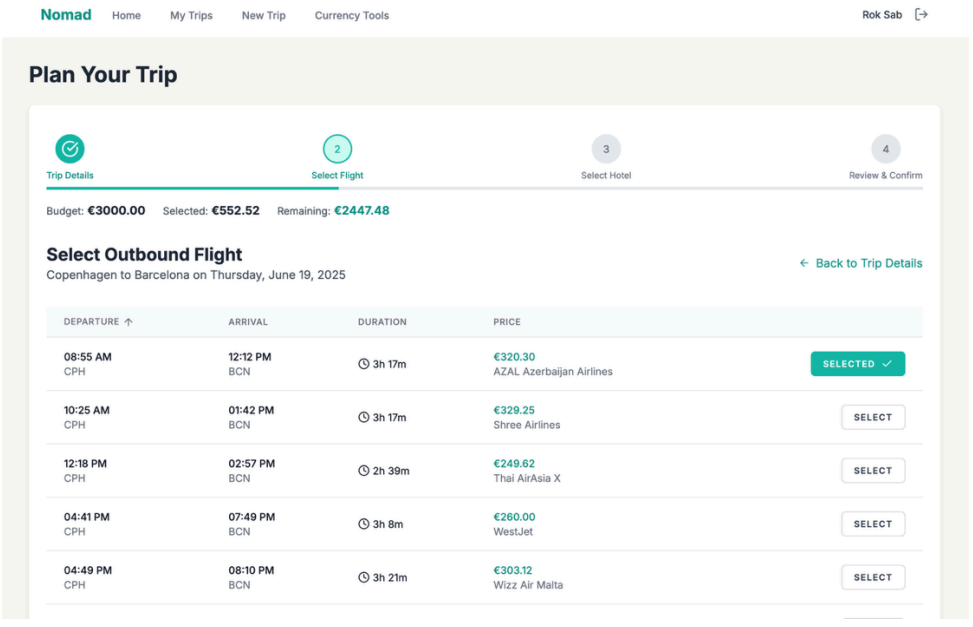
Pritiskom na *New trip*, korisnik dobiva formu za novo putovanje.



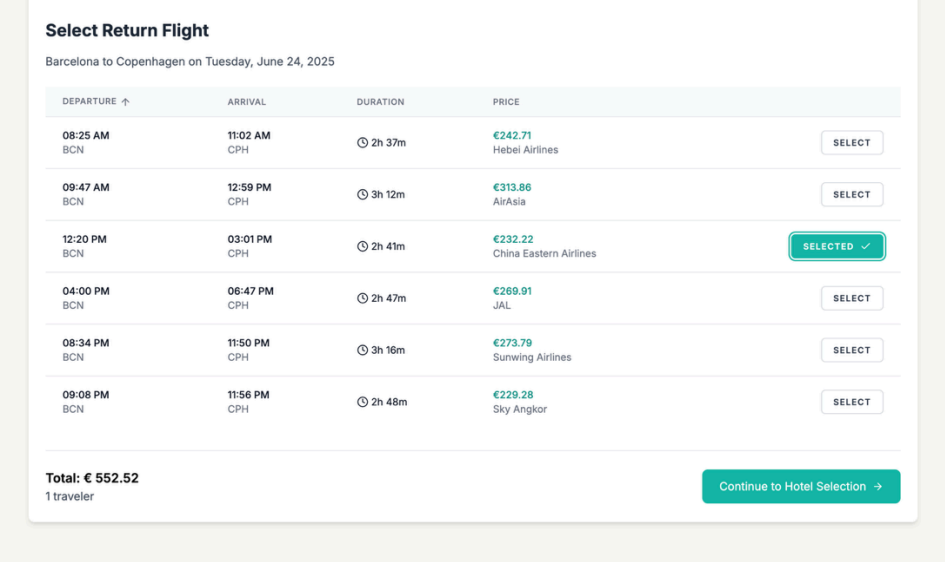
Slika 4.6 - Prvi korak kreiranja novog putovanja

Prvi korak novog putovanja sastoji se od osnovnih detalja: ime putovanja, broj putnika, mjesto polaska i odredište, željeno vrijeme putovanja te budžet korisnika.

Klikom na gumb *Continue to Flight Selection* korisnik prelazi na sljedeći korak.



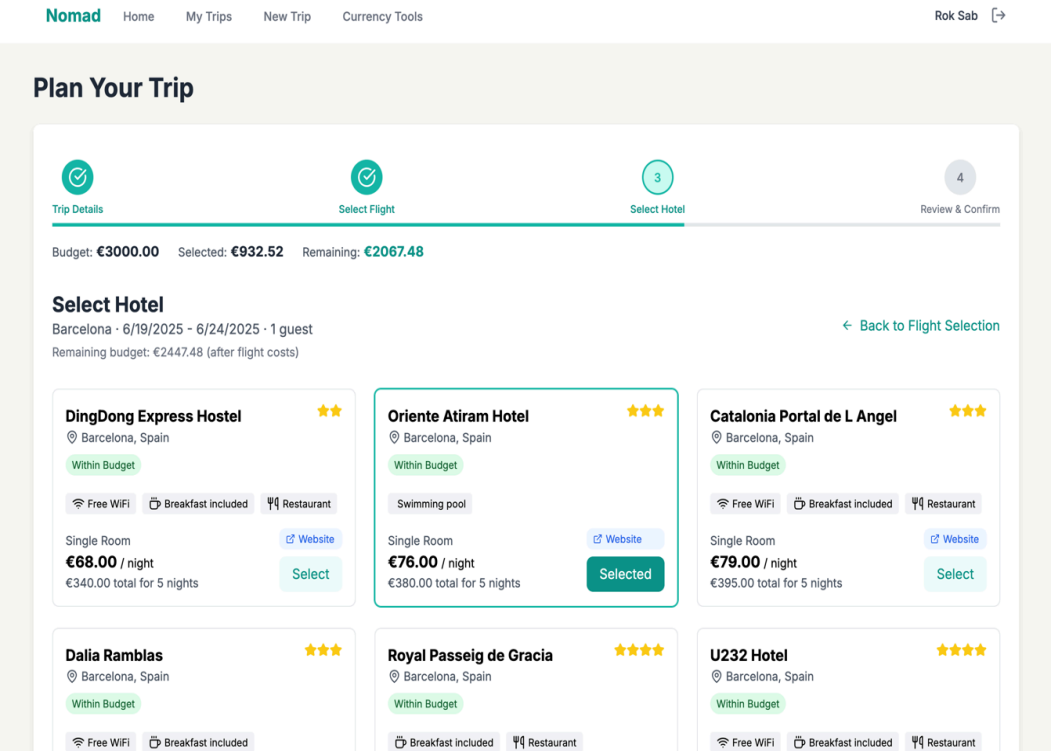
Slika 4.7 - Odabir leta prema destinaciji



Slika 4.8 - Odabir povratnog leta

U sljedećem koraku korisnik bira let za destinaciju i povratni let. Nude mu se brojne opcije letova s informacijama o vremenu polaska, dolaska, trajanju, cijeni i aviokompaniji.

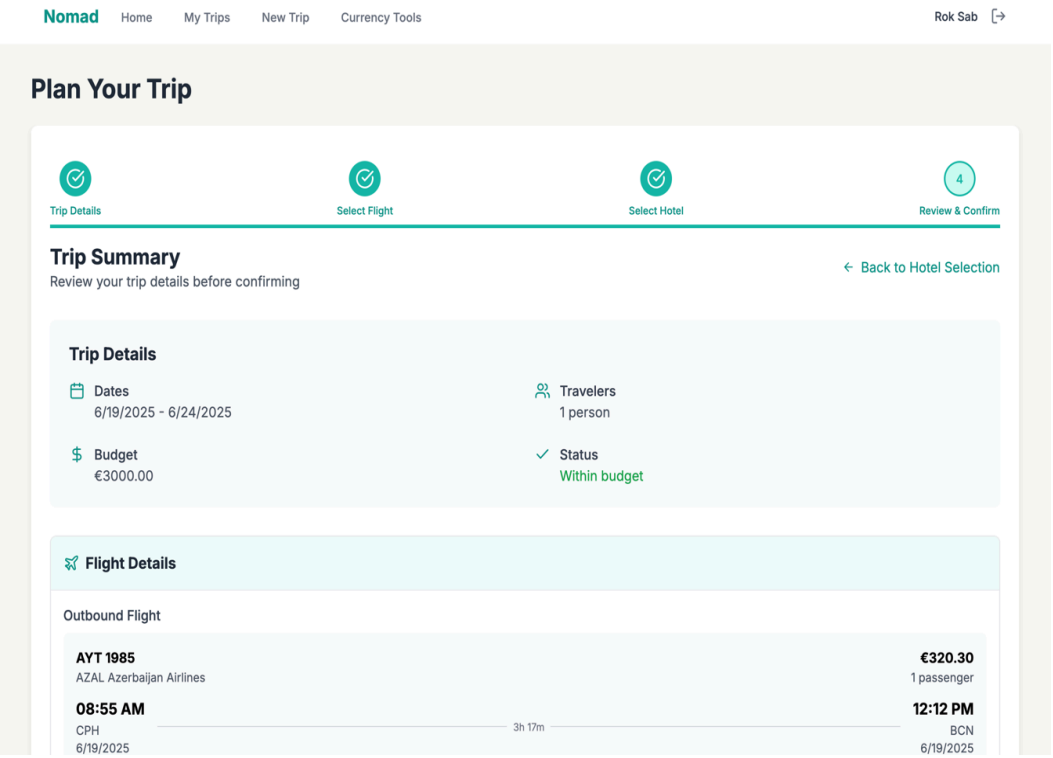
Pritiskom na gumb *Continue to Hotel Selection* korisnik prelazi na idući korak.



Slika 4.9 - Odabir hotela

Treći korak u procesu kreiranja novog putovanja uključuje odabir hotela u kojem će korisnik boraviti. Korisniku se prikazuje deset hotela, pri čemu su neki cjenovno pristupačniji, a neki skuplji.

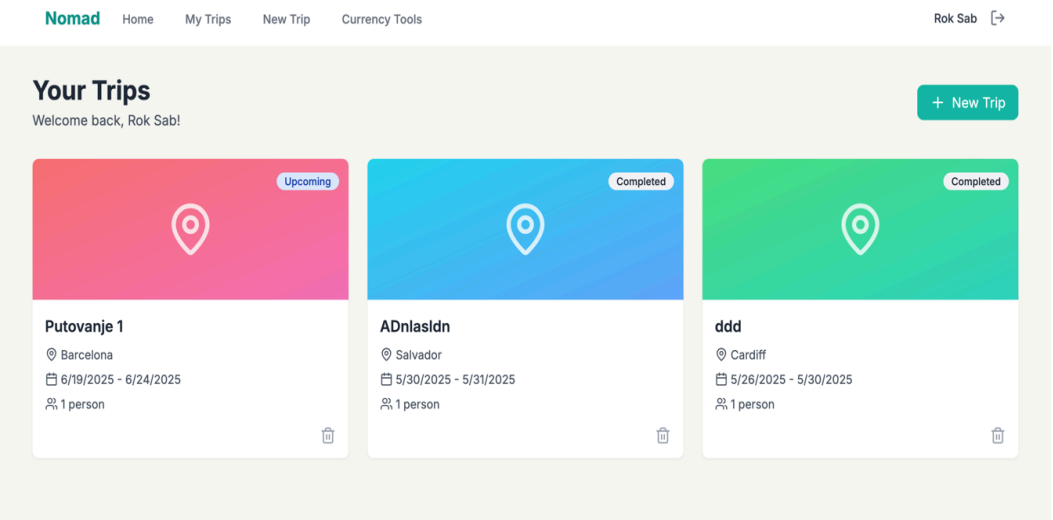
Za svaki hotel dostupni su sljedeći podaci: naziv, broj zvjezdica, lokacija, oznaka koja pokazuje nalazi li se hotel unutar korisnikova budžeta, cijena po noćenju, ukupna cijena boravka, poveznica na službenu internetsku stranicu hotela te popis sadržaja i usluga koje hotel nudi.



Slika 4.10 - Posljednji korak kreiranja putovanja

Završni korak u procesu kreiranja putovanja prikazuje sve relevantne detalje na jednom mjestu, omogućujući korisniku da ih pregleda u cijelosti. Nakon pregleda, korisnik putovanje potvrđuje pritiskom na gumb pri dnu stranice kojim se putovanje konačno kreira.

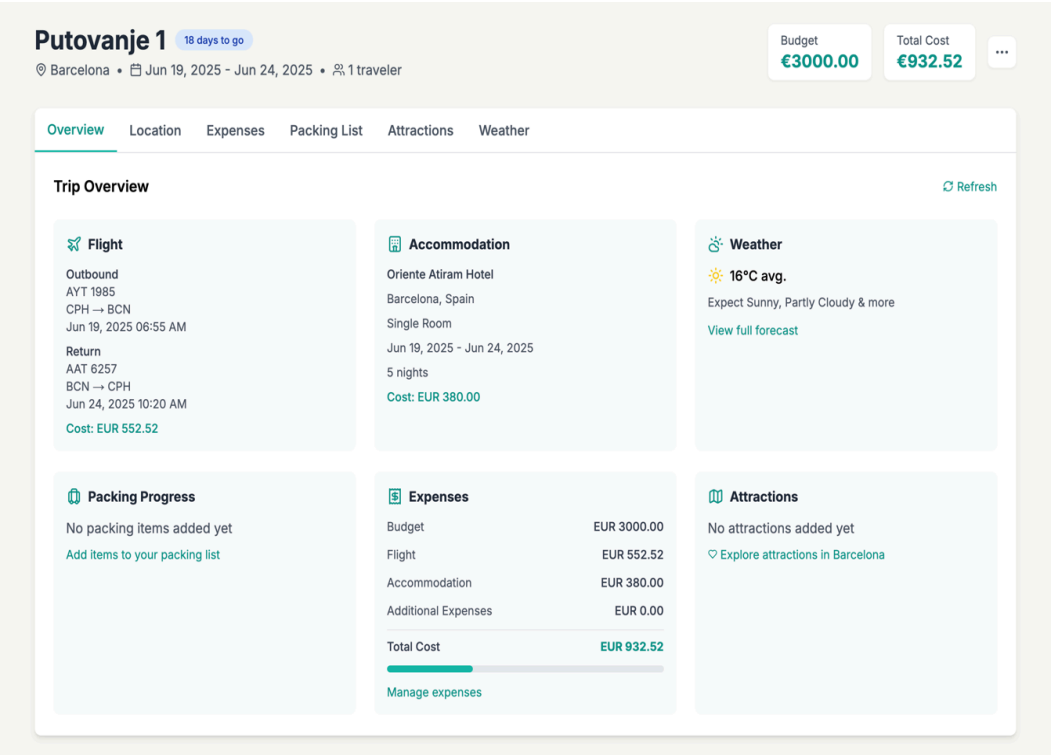
Pritiskom na gumb korisnik je preusmjeren na stranicu svih putovanja na kojoj se nalazi njegovo novo putovanje.



Slika 4.11 - Novo putovanje na listi korisnikovih putovanja

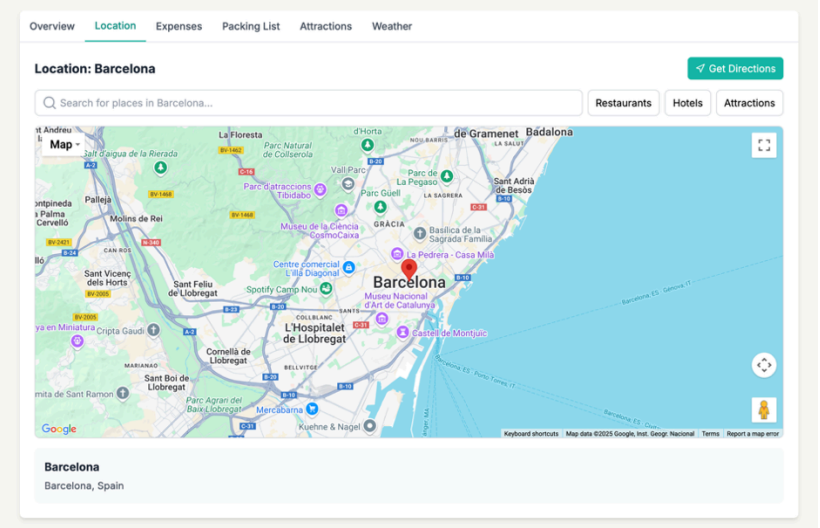
4.2.2. Pregled detalja putovanja

Klikom na pojedinačno putovanje u izborniku putovanja otvaraju se njegovi detalji.



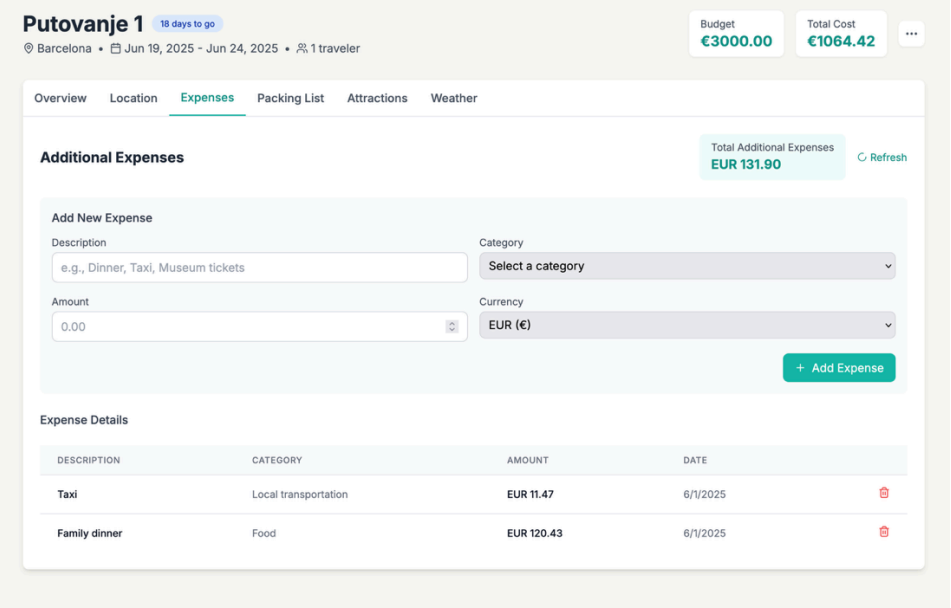
Slika 4.12 - Osnovni detalji putovanja

Prvi izbornik u detaljima putovanja nosi naziv *Overview*. U njemu su sažeto prikazane ključne informacije: podaci o letovima, smještaju, vremenskoj prognozi, popisu spakiranih stvari, troškovima te atrakcijama koje su korisniku privukle pažnju.



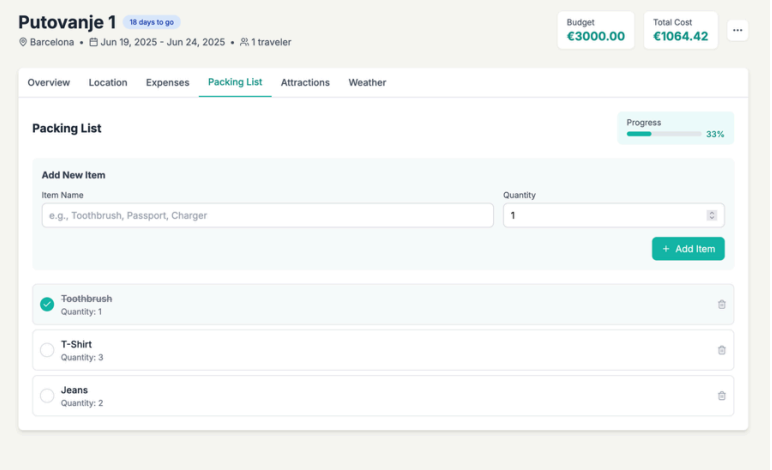
Slika 4.13 - Lokacija destinacije

Drugi izbornik, nazvan *Location*, prikazuje odredište putovanja pomoću sučelja Google Maps. Korisnik može pretraživati različita mjesta unutar grada te filtrirati prikaz na hotele, restorane ili atrakcije. Klikom na opciju *Get directions* korisnik se preusmjerava na Google Maps, gdje može pronaći detaljnije upute za dolazak do destinacije.



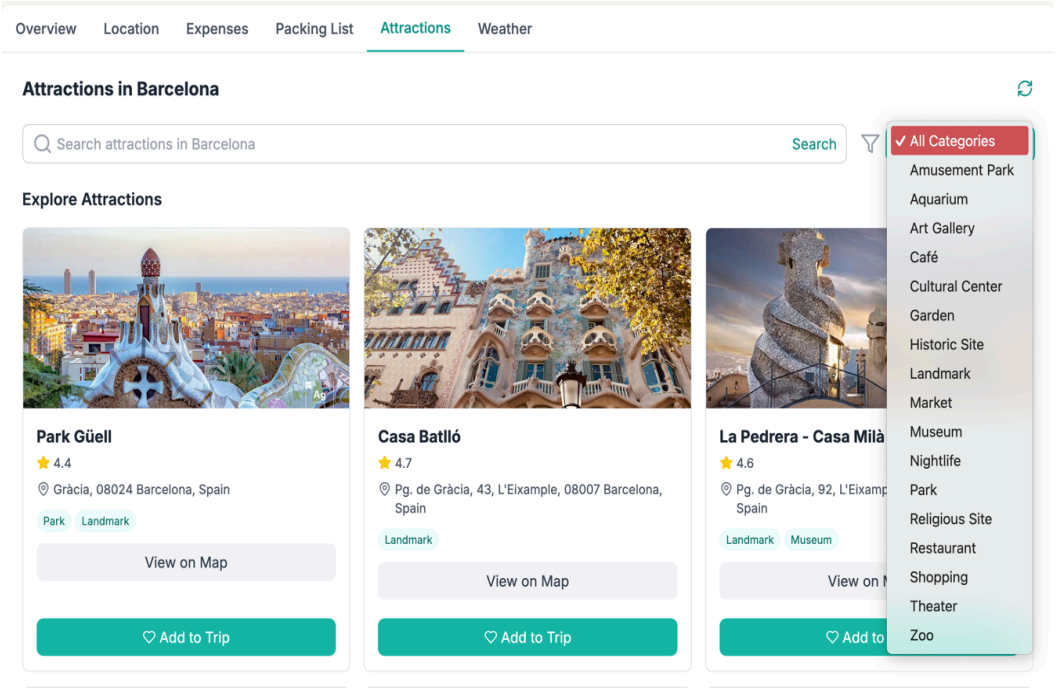
Slika 4.14 - Dodatni troškovi putovanja

Treći izbornik naziva se *Expenses*. U njemu korisnik može dodavati dodatne troškove putovanja koji se zbrajaju u ukupnu cijenu. Korisnik unosi opis troška, odabire kategoriju, cijenu i valutu. Na dnu stranice nalazi se pregled svih dodatnih troškova i njihovih datuma dodavanja.

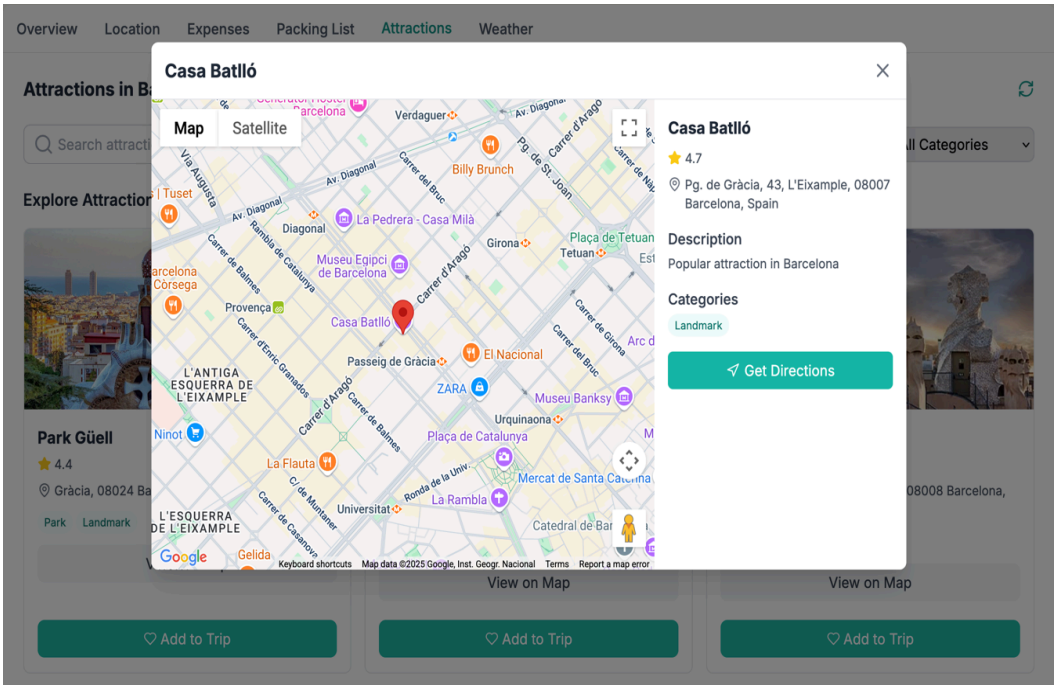


Slika 4.15 - Pakiranje stvari

Četvrti izbornik u detaljima putovanja naziva se Packing List, odnosno popis za pakiranje. U njemu korisnik može dodavati stavke koje želi ponijeti na put, uz mogućnost unosa količine za svaku od njih. Nakon toga, korisnik može označiti je li određenu stvar već spakirao ili nije.

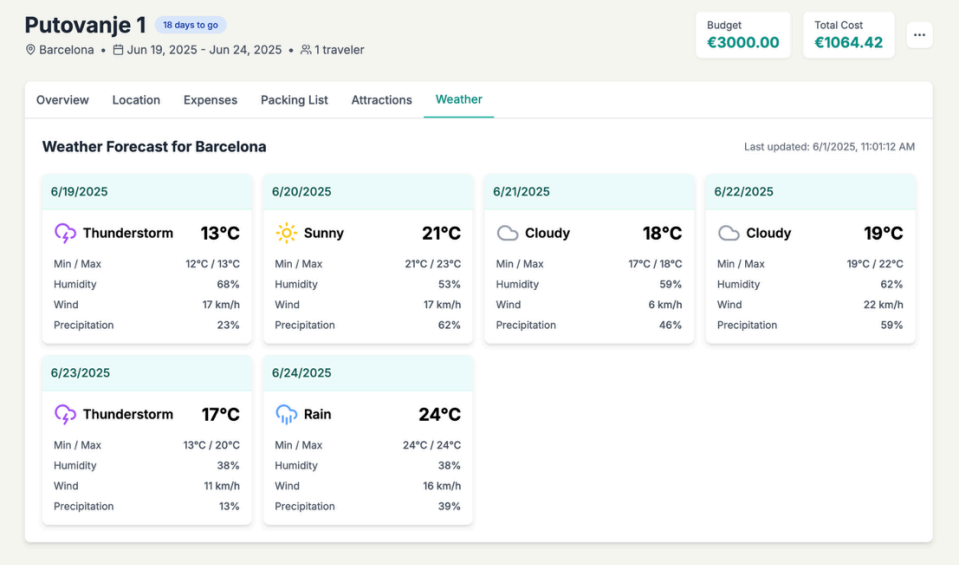


Slika 4.16 - Prikaz atrakcija i filtera



Slika 4.17 - Prikaz lokacije atrakcije

Peti izbornik u detaljima putovanja je *Attractions*, odnosno izbornik atrakcija. Ovdje korisnik ima pregled svih atrakcija za taj grad te ih može dodatno pretraživati ili filtrirati po različitim kategorijama. Za svaku atrakciju prikazani su ime, ocjena i adresa, a može se vidjeti i njezina lokacija na karti. Korisnik također može označiti koja mu se atrakcija sviđa te se ona dodaje u izbornik *Overview*.

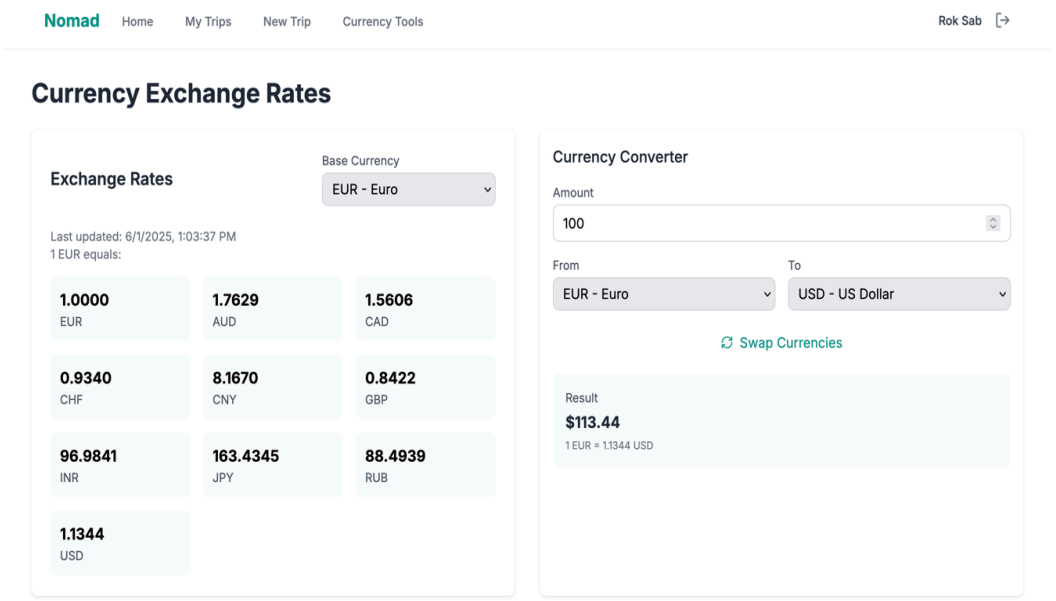


Slika 4.18 - Vremenska prognoza putovanja

Šesti izbornik je *Weather*, odnosno vremenska prognoza. Korisniku se na ovom izborniku prikazuje vremenska prognoza za svaki dan na putovanju. Prikazana je općenita temperatura, maksimalna i minimalna temperatura, vlaga, vjetar i oborine.

4.3. Tečajevi valuta

U navigacijskoj traci klikom na *Currency Tools* korisnik dolazi do izbornika za prikaz tečajeva i konverziju valuta.



Slika 4.19 - Izbornik za valute

Na lijevom dijelu izbornika korisnik može odabrati za koju valutu želi vidjeti trenutno stanje i konverziju.

Na desnom dijelu izbornika korisnik može izvršiti konverziju neke novčane svote iz jedne valute u neku drugu.

5. KORIŠTENE TEHNOLOGIJE

U idućih nekoliko odlomaka opisane su korištene tehnologije za izradu ovog projekta.

5.1. Next.js

Next.js [7] je moderni React okvir (engl. framework) otvorenog koda koji omogućuje izgradnju bržih i skalabilnih web-aplikacija. Podržava server-side rendering, statičku generaciju stranica te dinamičko generiranje sadržaja, čime omogućuje optimizirane performanse i poboljšanu SEO vidljivost. U ovom projektu koristi se kao temelj frontend dijela aplikacije.

5.2. Vercel

Vercel [8] je cloud platforma za automatsko postavljanje (deployment) i hostanje web-aplikacija. Namijenjen je jednostavnoj integraciji s Next.js projektima te omogućuje CI/CD tijek rada. U ovom radu koristi se za automatsko postavljanje i hosting aplikacije bez potrebe za ručnom konfiguracijom infrastrukture.

5.3. TypeScript

TypeScript [9] je nadskup JavaScript jezika koji uvodi statičko tipiziranje. Korištenjem tipova poboljšava se pouzdanost i održivost koda, osobito u većim projektima. U ovom projektu TypeScript se koristi radi unapređenja sigurnosti i čitljivosti frontend logike.

5.4. Tailwind CSS

Tailwind CSS [10] je utility-first CSS okvir koji omogućuje brzo oblikovanje korisničkog sučelja pomoću unaprijed definiranih klasa. Time se izbjegava pisanje vlastitih stilskih pravila, što ubrzava razvoj sučelja i osigurava dosljedan dizajn.

5.5. Supabase

Supabase [11] je backend platforma otvorenog koda koja nudi funkcionalnosti slične Firebaseu. Temelji se na PostgreSQL-u i omogućuje autentifikaciju, pohranu podataka te kreiranje RESTful API-ja i API-ja za rad u stvarnom vremenu bez dodatne konfiguracije. U ovom radu koristi se kao baza podataka i servis za autentifikaciju korisnika.

5.6. Visual Studio Code

Visual Studio Code [12] je lagani i proširivi uređivač izvornog kôda koji je razvio Microsoft. Omogućuje podršku za TypeScript, JavaScript, te integraciju s alatima poput Gita, ESLinta i Supabase dodatka. Koristi se kao glavno razvojno okruženje u projektu.

5.7. GitHub

GitHub [13] je platforma za verzioniranje kôda temeljena na Git sustavu. Omogućuje kolaboraciju, upravljanje granama, izvještavanje o pogreškama i CI/CD procese. U sklopu ovog rada GitHub se koristi za upravljanje verzijama izvornog koda.

5.8. Kaggle

Kaggle [14] je platforma za znanstvenu obradu podataka i strojno učenje, poznata po otvorenim skupovima podataka i natjecanjima. U ovom radu koristi se kao izvor podataka o aerodromima i hotelima, koji se kasnije čiste Python bibliotekama i integriraju u aplikaciju.

5.9. Pandas (Python)

Pandas [15] je moćna biblioteka za obradu i analizu podataka u programskom jeziku Python. Omogućuje rad s tabelarnim podacima putem objekta DataFrame, koji pruža napredne metode za filtriranje, agregaciju, sortiranje i čišćenje podataka. U okviru ovog projekta Pandas se koristi za obradu podataka preuzetih s platforme Kaggle, uključujući čišćenje nekonzistentnih zapisa, konverziju formata (npr. iz strukturirane tablice u JSON) te pripremu podataka za unos u bazu. Ova biblioteka znatno ubrzava obradu većih količina podataka i smanjuje rizik od pogrešaka u ručnoj manipulaciji.

5.10. Google Places API

Google Places API [16] omogućava pristup informacijama o mjestima (lokacijama), kao što su gradovi, znamenitosti, restorani i druge točke interesa. U kontekstu ovog projekta, koristi se za dohvaćanje detalja o atrakcijama na odabranoj destinaciji. Korisniku se na temelju unesene lokacije prikazuje lista atrakcija s pripadajućim podacima poput naziva, opisa, fotografija, zemljopisne širine i dužine, ocjena te vanjskih poveznica. API se također koristi za određivanje trenutačne lokacije korisnika, odnosno grada u kojem se nalazi, čime se omogućuje dinamično prikazivanje sadržaja vezanog uz to područje. Ova funkcionalnost znatno povećava interaktivnost i korisničku relevantnost prikazanih podataka.

5.11. ExchangeRate API

ExchangeRate API [17] je web servis koji omogućuje dohvaćanje najnovijih deviznih tečajeva između različitih valuta. U sklopu aplikacije koristi se za prikaz trenutačnih tečajeva korisniku, kao i za automatsku konverziju valuta prilikom unosa troškova. Na primjer, korisnik može unositi troškove u lokalnoj valuti zemlje u koju putuje, dok sustav automatski preračunava iznos u osnovnu valutu (npr. EUR). Time se omogućuje precizno praćenje budžeta bez potrebe za ručnim izračunima i smanjuje potencijalna greška prilikom financijskog planiranja putovanja.

ZAKLJUČAK

Ova aplikacija, osmišljena kao pomoćnik za cjelovito planiranje putovanja, uspješno je ostvarenje koje koristi suvremene internetske tehnologije. Cilj je bio stvoriti program koji je istovremeno bogat mogućnostima i jednostavan za korištenje, a ovaj rad pokazuje kako je taj cilj postignut spajanjem pouzdane pozadinske osnove s preglednim korisničkim sučeljem.

Srce aplikacije, njezin pozadinski dio, izgrađen je naprednim metodama koje povezuju korisnički dio s uslugama za obradu i čuvanje podataka. Ključnu ulogu ima vanjska usluga (Supabase) koja osigurava sigurnu pohranu svih informacija – od detalja o putovanjima i korisnicima do podataka o letovima, smještaju i znamenitostima – te pouzdan sustav za prijavu korisnika. Takav pristup ubrzava izradu i olakšava buduća proširenja. Aplikacija se također sigurno povezuje s vanjskim izvorima informacija, poput servisa za podatke o turističkim odredištima ili tečajeve valuta.

Izgled i način korištenja aplikacije, odnosno korisničko sučelje, stvoreni su pomoću suvremenih alata koji omogućuju preglednu i logičnu organizaciju. Korištenjem modernih tehnika za oblikovanje postignut je privlačan i prilagodljiv dizajn, prikladan za različite uređaje. Podaci se unutar aplikacije obrađuju na učinkovit način, osiguravajući ugodno korisničko iskustvo. Pažljiv odabir programskih jezika i alata pridonio je pouzdanosti i lakšem održavanju cjelokupnog sustava.

Kroz izradu ove aplikacije pokazano je kako se spajanjem suvremenih pristupa u izradi internetskih programa može stvoriti napredan alat koji odgovara stvarnim potrebama korisnika. Od početnog unosa podataka o putovanju, preko pomoći pri odabiru prijevoza i smještaja, pa sve do upravljanja troškovima i istraživanja odredišta, aplikacija nudi sveobuhvatnu podršku, pojednostavljujući inače složen proces organizacije putovanja.

Ovaj projekt ne samo da ispunjava svoje zadane funkcionalnosti, već predstavlja i primjer dobro strukturiranog i tehnološki suvremenog rješenja. Stečena znanja i iskustva u njegovoj izradi dragocjena su za razumijevanje gradnje modernih internetskih sustava, a sama aplikacija spremna je za daljnja poboljšanja i razvoj novih mogućnosti.

LITERATURA

[1] "TripIt", Poveznica: https://www.tripit.com/, Pristupljeno: 27. travnja 2025.

[2] Slika 1.1 – Dodavanje putovanja u TripIt (izvor [1])

[3] "Google Travel", Poveznica: https://www.google.com/travel/, Pristupljeno: 15. svibnja 2025.

[4] Slika 1.2 – Google Travel sučelje za letove (izvor [3])

[5] "Wanderlog", Poveznica: https://wanderlog.com/, Pristupljeno: 9. travnja 2025.

[6] Slika 1.3 – Prikaz putovanja na Wanderlog sučelju (izvor [5])

[7] "Next.js", Poveznica: https://nextjs.org/, Pristupljeno: 18. ožujka 2025.

[8] "Vercel", Poveznica: https://vercel.com/, Pristupljeno: 18. ožujka 2025.

[9] "TypeScript", Poveznica: https://www.typescriptlang.org/, Pristupljeno: 18. ožujka 2025.

[10] "Tailwind CSS", Poveznica: https://tailwindcss.com/, Pristupljeno: 18. ožujka 2025.

[11] "Supabase", Poveznica: https://supabase.com/, Pristupljeno: 30. ožujka 2025.

[12] "Visual Studio Code", Poveznica: https://code.visualstudio.com/, Pristupljeno: 10. ožujka 2025.

[13] "GitHub", Poveznica: https://github.com/, Pristupljeno: 6. travnja 2025.

[14] "Kaggle", Poveznica: https://www.kaggle.com/, Pristupljeno: 14. travnja 2025.

[15] "Pandas", Poveznica: https://pandas.pydata.org/, Pristupljeno: 10. ožujka 2025.

[16] "Google Places API", Poveznica: https://developers.google.com/maps/documentation/places/web-service, Pristupljeno: 3. svibnja 2025.

[17] "ExchangeRate API", Poveznica: https://www.exchangerate-api.com/, Pristupljeno: 29. travnja 2025.

SAŽETAK

Naslov: Personalizirana aplikacija za planiranje putovanja

Ova personalizirana aplikacija za planiranje putovanja osmišljena je kako bi korisnicima pružila jedinstveno i prilagođeno iskustvo organizacije putovanja. Fokusirajući se na individualne preferencije, interese i budžet, aplikacija nudi pametne preporuke za odredišta, aktivnosti, smještaj i prijevoz. Kroz intuitivno sučelje, korisnici mogu jednostavno kreirati detaljne planove puta, upravljati rezervacijama, pratiti troškove te otkrivati skrivene dragulje prilagođene upravo njima, čineći svako putovanje istinski osobnim doživljajem.

Ključne riječi: putovanje, web aplikacija, atrakcije, Next.js, TypeScript, PostgreSQL

SUMMARY

Title: Personalized travel planning application

This personalized travel planning application is designed to provide users with a unique and tailored experience for organizing their trips. Focusing on individual preferences, interests, and budget, the application offers smart recommendations for destinations, activities, accommodation, and transportation. Through an intuitive interface, users can easily create detailed itineraries, manage bookings, track expenses, and discover hidden gems customized just for them, making every journey a truly personal experience.

Keywords: travel, web application, attractions, Next.js, TypeScript, PostgreSQL

---

1. Mock podaci su lažni ili simulirani podaci koji se koriste za testiranje i razvoj prije nego što se poveže stvarni izvor podataka ili API.↩